

TRABAJO DE FIN DE GRADO

Grado en Ingeniería de Computadores

ESCUELA POLITÉCNICA SUPERIOR

Diseño y modificación de entornos urbanos en CARLA
para la simulación de situaciones viales realistas y su
análisis semántico mediante modelos de lenguaje LLM

Gabriel Hernández Velandia

Tutor: Augusto Luis Ballardini

2026

UNIVERSIDAD DE ALCALÁ

ESCUELA POLITÉCNICA SUPERIOR

Grado en Ingeniería de Computadores

Trabajo de Fin de Grado

**Diseño y modificación de entornos urbanos en CARLA para la
simulación de situaciones viales realistas y su análisis semántico
mediante modelos de lenguaje LLM**

Autor: Gabriel Hernández Velandia

Tutor: Augusto Luis Ballardini

Tribunal:

Presidente: Iván García Daza

Vocal: Melquíades Carbajo Martín

Suplente: Javier Pedro Carracedo

Fecha de depósito: 10 de julio de 2026

Agradecimientos

La realización de este trabajo ha sido posible gracias al apoyo, la confianza y el acompañamiento de las personas que han estado a mi lado a lo largo de este camino.

En primer lugar, quiero agradecer a mis padres y a mi hermana por su apoyo incondicional, por estar siempre presentes en los momentos más difíciles y por no dejar nunca de creer en mí. Su confianza y ánimo han sido fundamentales para llegar hasta aquí.

También quiero dar las gracias a mis amigos, por acompañarme durante este proceso, por su apoyo y por interesarse siempre por cómo avanzaba el trabajo. Su cercanía y sus palabras de ánimo han hecho este camino mucho más llevadero.

Por último, quiero expresar un agradecimiento especial a mi tutor, Augusto, por su dedicación, paciencia y apoyo constante durante el desarrollo de este proyecto. Su orientación, disponibilidad y confianza han sido esenciales para que este trabajo pudiera llevarse a cabo de la mejor manera posible.

Resumen

Este Trabajo de Fin de Grado propone el diseño y modificación de entornos urbanos destinados a la simulación de vehículos autónomos en CARLA. Se emplean mapas predeterminados y escenarios generados a partir de datos de Open Street Map (OSM) mediante su edición en RoadRunner y Unreal Engine, incorporando obstáculos y restricciones de circulación. Sobre estos escenarios se ejecutan rutas de prueba cuyos recorridos son capturados mediante cámaras virtuales. Los vídeos obtenidos se analizan mediante Modelos de Lenguaje a Gran Escala Large Language Model (LLM) para identificar bloqueos y describir semánticamente el entorno. Finalmente, el rendimiento del modelo se evalúa mediante métricas Precision y Recall.

Palabras clave: CARLA, Simulación de vehículos autónomos, diseño de entornos urbanos, Análisis semántico, Modelos de Lenguaje a Gran Escala.

Abstract

This Final Degree Project proposes the design and modification of urban environments for simulating autonomous vehicles in CARLA. Predefined maps and scenarios generated from OpenStreetMap (OSM) data are used, edited in RoadRunner and Unreal Engine to incorporate obstacles and traffic restrictions. Test routes are run on these scenarios, and their paths are captured using virtual cameras. The resulting videos are analyzed using Large Language Models (LLMs) to identify blockages and semantically describe the environment. Finally, the model's performance is evaluated using Precision and Recall metrics.

Keywords: CARLA, Autonomous vehicle simulation, urban environment design, semantic analysis, large-scale language models.

Resumen extendido

Este Trabajo de Fin de Grado presenta una metodología para el diseño y modificación de entornos urbanos destinados a la simulación de vehículos autónomos en el simulador CARLA. El objetivo principal es generar escenarios realistas y variados que reproduzcan situaciones habituales presentes en entornos urbanos reales, incorporando elementos que puedan alterar la circulación normal de los vehículos, tales como obras, cierres de vías, barreras, obstáculos y otras incidencias que requieran la adopción de rutas alternativas.

La creación de estos escenarios se aborda mediante dos estrategias complementarias. Por un lado, se modifican mapas predeterminados incluidos en el simulador CARLA, conocidos como *Towns*, seleccionando aquellos que presentan una mayor riqueza geométrica y diversidad vial entre los distintos escenarios disponibles. Por otro lado, se desarrollan nuevos entornos a partir de datos geográficos extraídos de [OSM](#), una herramienta que proporciona información cartográfica global a partir de la cual es posible seleccionar manualmente áreas de interés o utilizar conjuntos de datos previamente preparados por la comunidad, correspondientes a grandes ciudades como Madrid, Barcelona, Houston o Berlín, entre otras.

Una vez extraídos, estos datos geográficos son procesados y editados mediante la herramienta RoadRunner con el objetivo de generar redes viales tridimensionales. Esta herramienta permite ajustar distintos parámetros en función del comportamiento que se desea analizar en las simulaciones posteriores, como por ejemplo, límites de velocidad, prioridades de paso o maniobras de circulación. Las redes generadas luego son importadas en el entorno base sobre el que está construido el simulador CARLA, el motor Unreal Engine, donde las redes mencionadas se enriquecen con elementos físicos y visuales que proporcionan un contexto más realista para el análisis siguiente mediante modelos de lenguaje.

Adicionalmente, los elementos físicos que actúan como bloqueos o restricciones se generan a partir de un archivo de configuración en formato YAML, que contiene información sobre el tipo de obstáculo y su posición dentro del entorno. Este archivo es interpretado mediante scripts en Python a través de la Application Programming Interface (API) de CARLA, lo que permite automatizar la inserción de dichos elementos en los escenarios.

Una vez construidos los escenarios, se ejecutan rutas específicas previamente definidas tanto para los entornos generados como para los predeterminados. Estas rutas se clasifican en distintos niveles de dificultad en función de parámetros como la presencia de obstáculos, la distancia recorrida o la existencia de bucles de circulación, es decir, el recorrido continuo de una misma intersección. Durante su ejecución, se capturan secuencias de imágenes mediante una cámara virtual integrada en el vehículo a través de la [API](#) de CARLA, posicionada de forma que permita registrar de manera adecuada el entorno percibido por el vehículo en todo momento. Estas capturas se procesan posteriormente para generar grabaciones completas del recorrido realizado.

Los vídeos obtenidos son analizados mediante un Modelos de Lenguaje a Gran Escala [LLM](#). Para ello, se diseña un *prompt* especializado que define las reglas, restricciones y criterios de análisis que debe

seguir el modelo con el objetivo de generar una descripción semántica lo más precisa posible del entorno. El objetivo principal del análisis consiste en identificar elementos que supongan bloqueos o restricciones para la circulación y que obliguen al vehículo a modificar su trayectoria, considerando tanto la eficiencia del recorrido como los criterios de seguridad vial.

Finalmente, se evalúa el rendimiento del modelo mediante métricas clásicas de recuperación de información, como *Precision* y *Recall*, con el propósito de determinar su capacidad para detectar correctamente los elementos bloqueantes presentes en los escenarios simulados. Los resultados obtenidos permiten valorar el potencial de los [LLM](#) como herramienta de apoyo para el entrenamiento, validación y mejora de la robustez de sistemas de conducción autónoma en entornos urbanos complejos.

Índice

Agradecimientos	v
Resumen	vii
Abstract	ix
Resumen extendido	xi
Índice	xiii
Índice de figuras	xvii
Índice de tablas	xix
Lista de acrónimos	xxi
Lista de símbolos	xxi
1 Introducción	1
1.1 Contexto	1
1.2 Motivación	2
1.3 Objetivo	2
2 Estudio teórico	5
2.1 Introducción	5
2.2 Estado del Arte	5
2.2.1 Conducción Autónoma	5
2.2.1.1 Niveles de clasificación de conducción autónoma	6
2.2.2 Simulación para validación vehículos autónomos	7
2.2.3 Principales simuladores para conducción autónoma	8
2.2.4 Modelos de lenguaje aplicados a la visión	10
2.2.4.1 Evolución de los LLM	10
2.2.4.2 Modelos de Lenguaje Visual	12
2.2.4.3 Aplicación de los modelos de lenguaje visual a la conducción autónoma	14
3 Metodología y herramientas empleadas	17
3.1 Introducción	17
3.2 Simulador CARLA	17
3.2.1 Arquitectura del simulador	18
3.2.2 Motor gráfico: Unreal Engine	19
3.2.3 ASAM OpenDRIVE	20

3.2.4	Mapas del simulador	21
3.2.5	Sistema de Sensores	23
3.2.6	API de Python	24
3.3	OpenStreetMap	26
3.4	RoadRunner	28
4	Diseño y ejecución de escenarios urbanos predeterminados	31
4.1	Introducción	31
4.2	Preparación de los escenarios predeterminados	31
4.2.1	Selección de escenarios	31
4.2.2	Definición de rutas	33
4.2.3	Selección de obstáculos	35
4.3	Configuraciones de los escenarios	37
4.3.1	Diseño del archivo de configuración	38
4.3.2	Generación dinámica de los obstáculos	39
4.4	Configuración del vehículo autónomo	41
4.4.1	Waypoints	41
4.4.2	Configuraciones Traffic Manager	42
4.5	Ejecución de las simulaciones	43
4.6	Obtención de videos	45
5	Generación de escenarios urbanos personalizados	47
5.1	Introducción	47
5.2	Obtención de datos cartográficos	47
5.2.1	Selección del área urbana	47
5.2.2	Exportación de datos geográficos	48
5.3	Construcción base del escenario mediante RoadRunner	49
5.3.1	Importación del mapa base	49
5.3.2	Configuración de la red vial	50
5.3.3	Exportación del escenario	52
5.4	Adaptación del escenario en Unreal Engine	53
5.4.1	Importación en Unreal Editor	53
5.4.2	Ajustes del entorno	54
5.5	Validación de los escenarios generados	55
6	Análisis de escenarios mediante modelos LLM	57
6.1	Introducción	57
6.2	Preparación de los datos	57
6.2.1	Formato de entrada de los modelos	57
6.2.2	Diseño del prompt	59
6.3	Modelos multimodales evaluados	60
6.3.1	Qwen2.5-VL	61
6.3.2	LLaVA 1.6	61
6.3.3	InternVL3	62
6.3.4	Qwen3-VL	62
6.4	Evaluación realizada sobre los modelos	63
6.4.1	Diseño de las pruebas	63
6.5	Resultados obtenidos	64

6.5.1	Evaluación preliminar de los modelos	64
6.5.2	Evaluación completa del modelo seleccionado	65
6.5.3	Análisis cualitativo	66
6.6	Interpretación de los resultados	67
7	Conclusiones y líneas futuras	69
7.1	Conclusiones	69
7.2	Líneas futuras	70
	Bibliografía	73
	Apéndice A Detalles de implementación	75
A.1	Comandos de ejecución	75
A.2	Archivos de configuración y archivos de LLM	75
A.3	Prompts utilizados para los modelos probados	78
	Apéndice B Herramientas y recursos	81

Índice de figuras

2.1	Evolución de los LLM. [16]	12
2.2	Flujo de un Visual-Language Model (VLM)	14
3.1	Arquitectura Cliente-Servidor de CARLA	18
3.2	Ejemplo de la interfaz base de Unreal Engine y la simulación de CARLA ejecutándose sobre este.	20
3.3	Definición de carriles contenido en archivo .xodr	20
3.4	Vistas de diferentes ángulos de mapa predeterminado Town02	22
3.5	Vistas de diferentes ángulos de mapa predeterminado Town05.	23
3.6	Vistas de diferentes ángulos de mapa predeterminado Town06.	23
3.7	Vistas de diferentes ángulos de mapa predeterminado Town10.	23
3.8	Interfaz de selección de entorno urbano OpenStreetMap	27
4.1	Mapas predeterminados elegidos del simulador CARLA.	33
4.2	Comparación entre ruta óptima y ruta alterada por obstáculos (Town02)	34
4.3	Comparación entre ruta óptima y ruta alterada por obstáculos (Town03).	34
4.4	Mapas de CARLA clasificados según dificultad.	35
4.5	Catálogo de objetos más utilizados para el bloqueo de rutas.	36
4.6	Ejemplo de <i>props</i> adicionales a los predeterminados de CARLA.	37
4.7	Estructura jerárquica del archivo de configuración YAML.	39
4.8	Diagrama de flujo del proceso de generación de bloqueos.	40
4.9	Comparativa entre calles sin bloqueo y calles inhabilitadas luego de generación de <i>props</i> .	40
4.10	<i>Waypoints</i> guía generados para establecimiento de rutas.	41
4.11	Establecimiento de ruta sobre <i>Waypoints</i> guía.	42
4.12	Interfaz gráfica para la selección de escenarios y bloqueos.	44
4.13	Diagrama de flujo del proceso de ejecución de una simulación hasta la obtención de videos.	44
4.14	Field Of View (FOV) de vehículo utilizado para la captura de imágenes.	45

5.1	Obtención de los datos geográficos para la generación de escenarios.	48
5.2	Previsualización de la red de carreteras contenidas en el archivo .osm importado en RoadRunner.	49
5.3	Previsualización de la red de carreteras luego de limpieza.	50
5.4	Entorno generado en base al archivo .osm con la herramienta SD Builder.	51
5.5	Antes y después de aplicar correcciones a inconsistencias generadas.	51
5.6	Entorno cargado dentro de Unreal Editor.	54
5.7	Conjunto de tomas de diferentes ángulos de los escenarios poblados.	55
5.8	Validación de funcionamiento de escenarios con la aparición de <i>Waypoints</i> en los entornos personalizados.	56
6.1	Flujo de procesamiento del vídeo para la inferencia con LLM.	58
6.2	Ejemplo de imagen completa y sus tres secciones codificadas y ampliadas.	59
6.3	Pruebas de identificación de obstáculos del modelo elegido.	66

Índice de tablas

2.1	Comparativa entre simuladores.	9
3.1	Mapas incluidos en CARLA y características generales.	22
3.2	Principales sensores disponibles en CARLA y clasificación según su función.	24
3.3	Principales clases y módulos de la API de Python de CARLA utilizados en el desarrollo del proyecto.	26
3.4	Principales funcionalidades de RoadRunner Utilizadas.	29
4.1	Configuraciones principales aplicadas al Traffic Manager.	43
4.2	Parámetros de configuración de la cámara y del proceso de generación de vídeo.	46
5.1	Comprobaciones realizadas durante la validación del escenario generado.	56
6.1	Componentes principales del <i>prompt</i> empleado durante la evaluación.	60
6.2	Parámetros de inferencia utilizados para Qwen2.5-VL.	61
6.3	Parámetros de inferencia utilizados para InternVL3.	62
6.4	Parámetros de inferencia utilizados para Qwen3-VL.	63
6.5	Resumen de los escenarios utilizados durante la evaluación experimental.	63
6.6	Criterios empleados para la evaluación de los modelos.	64
6.7	Resultados obtenidos durante la evaluación preliminar de los modelos multimodales. . . .	65
6.8	Resultados de la evaluación completa realizada con Qwen3-VL.	66

Lista de acrónimos

ADS	Automated Driving System.
API	Application Programming Interface.
CARLA	Car Learning to Act.
CLIP	Contrastive Language-Image Pretraining.
FOV	Field Of View.
GPU	Graphics Processing Unit.
LLM	Large Language Model.
MLLM	Multimodal Large Language Models.
ODD	Operational Design Domain.
OSM	Open Street Map.
SAE	Society of Automotive Engineers.
VLM	Visual-Language Model.

Capítulo 1

Introducción

1.1 Contexto

La conducción autónoma se ha consolidado durante las últimas décadas como uno de los principales campos de investigación dentro del ámbito de la automoción. Este avance ha sido impulsado por el desarrollo de tecnologías relacionadas con la percepción artificial, el aprendizaje profundo y los sistemas de toma de decisiones, permitiendo la aparición de vehículos capaces de circular con una intervención humana mínima o incluso nula en determinados escenarios controlados. Sin embargo, el desarrollo de estos sistemas continúa representando un importante reto tecnológico debido a la complejidad inherente de los entornos urbanos y la gran diversidad de situaciones de tráfico que pueden presentarse en escenarios reales.

Entre los principales desafíos que aún limitan la adopción segura y generalizada de estas tecnologías destacan la presencia de obstáculos imprevistos, la necesidad de adaptarse a cambios en la infraestructura vial y la planificación dinámica de rutas. Situaciones como estas pueden obligar a recalcular rutas previamente establecidas para alcanzar el destino final, comprometiendo potencialmente la seguridad y la eficiencia del trayecto. En este contexto, resulta fundamental que los vehículos autónomos no solo sean capaces de percibir su entorno, sino también de interpretarlo semánticamente, analizarlo y tomar decisiones coherentes en función de las condiciones presentes, garantizando así una navegación segura y eficiente.

Paralelamente al desarrollo de los sistemas de conducción autónoma, los [LLM](#) han experimentado una evolución significativa. Los avances en hardware, especialmente en Graphics Processing Unit (GPU), junto con mejoras en las arquitecturas de aprendizaje profundo, las metodologías de entrenamiento y la incorporación de capacidades multimodales, han acelerado notablemente tanto el entrenamiento como la inferencia de estos modelos. Como resultado, actualmente existe una amplia variedad de [LLM](#) aplicados en ámbitos científicos, industriales y civiles, incluyendo tareas relacionadas con la visión por computador, el reconocimiento de elementos del entorno y el análisis semántico de información visual.

La combinación de estas capacidades con los sistemas de conducción autónoma abre nuevas líneas de investigación orientadas a mejorar la comprensión contextual de los escenarios de conducción. En este sentido, los [LLM](#) pueden actuar como sistemas de apoyo capaces de interpretar información visual compleja y proporcionar descripciones semánticas del entorno, identificando elementos relevantes como obstáculos, modificaciones en la infraestructura urbana o incidencias en la vía. Por ello, la integración de modelos de lenguaje multimodales en entornos de simulación representa una línea de investigación emergente con un gran potencial para el desarrollo de futuras soluciones de conducción autónoma.

1.2 Motivación

Con el objetivo de evaluar y mejorar los sistemas de percepción y toma de decisiones, la comunidad científica dedicada a la conducción autónoma ha recurrido en gran medida al uso de simuladores de conducción, los cuales permiten realizar pruebas en entornos controlados, reproducibles y seguros. Entre ellos, Car Learning to Act (CARLA) se ha consolidado como una de las plataformas de referencia en la investigación de conducción autónoma, llegando a considerarse un estándar de facto para el desarrollo y evaluación de sistemas basados en aprendizaje automático. [CARLA](#) es un simulador de código abierto diseñado como una plataforma modular y flexible que permite abordar múltiples tareas relacionadas con la conducción autónoma, incluyendo la percepción del entorno, la planificación de trayectorias, la navegación y la evaluación de algoritmos de visión por computador y aprendizaje automático.

A pesar de su amplia adopción en el ámbito de la investigación y de las numerosas funcionalidades que ofrece, diversos estudios han señalado que los escenarios disponibles en [CARLA](#) presentan ciertas limitaciones en términos de diversidad y realismo. Estas limitaciones pueden restringir la capacidad de generalización de los modelos desarrollados hacia contextos urbanos más complejos, dinámicos y representativos de situaciones reales.

Esta situación pone de manifiesto la necesidad de generar nuevos escenarios de simulación que permitan reproducir condiciones urbanas más variadas, incluyendo modificaciones en la infraestructura vial y la aparición de obstáculos o bloqueos que obliguen a alterar las rutas inicialmente previstas. Asimismo, resulta de interés explorar nuevas herramientas capaces de interpretar semánticamente estos escenarios y proporcionar información contextual sobre los eventos presentes en la vía. En este contexto, los recientes avances en modelos de lenguaje multimodales ofrecen una oportunidad para realizar análisis semánticos de las rutas y del entorno de conducción, permitiendo identificar incidencias, describir situaciones presentes en la vía y evaluar su posible impacto en la navegación autónoma. La necesidad de disponer de escenarios de simulación más representativos y de explorar nuevas capacidades de análisis semántico en entornos de conducción autónoma constituye la principal motivación para el desarrollo del presente trabajo.

1.3 Objetivo

El proyecto tiene como objetivo principal el diseño, desarrollo y modificación de entornos urbanos complejos, realistas y variados para su utilización en el simulador CARLA, con el propósito de reproducir situaciones habituales que pueden encontrarse en escenarios reales de conducción. Para ello, se plantea la modificación de mapas predeterminados incluidos en el simulador, así como la creación de nuevos entornos urbanos a partir de datos geográficos procedentes de [OSM](#). Estos escenarios incorporarán elementos como intersecciones alteradas, edificaciones, obras, cierres de vías y distintos tipos de obstáculos que permitan simular incidencias capaces de alterar el comportamiento esperado de los vehículos dentro de la simulación.

Asimismo, se persigue desarrollar un proceso de generación y enriquecimiento de escenarios que combine herramientas especializadas como RoadRunner, para la construcción de redes viales tridimensionales, y Unreal Engine, para la incorporación de elementos visuales y contextuales que incrementen el grado de realismo de las simulaciones. De esta forma, se pretende disponer de entornos suficientemente variados, tanto ficticios como existentes, que permitan evaluar situaciones de conducción más próximas a las condiciones presentes en entornos urbanos reales.

Una vez generados los escenarios, otro de los objetivos del trabajo consiste en ejecutar rutas previamente definidas mediante vehículos simulados y registrar la información visual obtenida durante los

recorridos. Estas secuencias servirán como base para la evaluación de una [LLM](#), con el fin de analizar su capacidad para interpretar semánticamente las escenas observadas, identificar obstáculos o restricciones presentes en la vía y describir aquellos elementos relevantes que puedan afectar a la navegación.

Finalmente, el proyecto persigue estudiar el potencial de los [LLM](#) como herramienta de apoyo en sistemas de conducción autónoma, evaluando su capacidad para reconocer incidencias presentes en los escenarios simulados y determinar su utilidad como mecanismo complementario tanto para el entrenamiento, validación y mejora de la robustez de futuros sistemas autónomos, así como para su integración en soluciones orientadas a escenarios reales de conducción.

Capítulo 2

Estudio teórico

2.1 Introducción

El desarrollo y la validación de sistemas de conducción autónoma y algoritmos de navegación urbana inteligente requieren entornos de prueba precisos, seguros y reproducibles. En este contexto, la simulación virtual se ha consolidado como una herramienta fundamental, permitiendo modelar escenarios complejos y situaciones de riesgo, como bloqueos viales, restricciones de circulación u obstáculos imprevistos, sin los costes económicos ni los riesgos asociados a las pruebas en entornos reales.

El presente estudio teórico tiene como objetivo analizar los fundamentos tecnológicos y las herramientas que sustentan el desarrollo de este proyecto. Para ello, se revisan los conceptos relacionados con la conducción autónoma, las plataformas de simulación empleadas en este ámbito, las técnicas de generación y modificación de entornos urbanos virtuales, así como el uso de [LLM](#) para el análisis semántico de información visual.

2.2 Estado del Arte

2.2.1 Conducción Autónoma

La conducción autónoma hace referencia a aquellos sistemas capaces de percibir el entorno, tomar decisiones y controlar un vehículo sin intervención directa del conductor. En los últimos años, esta tecnología ha experimentado una evolución significativa debido al avance de los sistemas de inteligencia artificial, el aumento de la capacidad computacional y la mejora de los sensores que los vehículos utilizan para realizar las detecciones y recolecciones de datos. Para estandarizar el nivel de autonomía de estos sistemas, la Society of Automotive Engineers (SAE) [\[1\]](#) define una escala que va desde el Nivel 0 (conducción completamente manual) hasta el Nivel 5 (automatización total en cualquier escenario).

Los esfuerzos de las investigaciones actuales se han centrado principalmente en alcanzar los niveles 4 y 5, donde el vehículo debe ser capaz de gestionar por sí mismo situaciones complejas sin el respaldo de un conductor humano.

Los primeros desarrollos de vehículos autónomos se basaban principalmente en sistemas deterministas compuestos por reglas definidas manualmente. Sin embargo, la incorporación de técnicas de aprendizaje automático y, especialmente, de aprendizaje profundo (Deep Learning), ha permitido desarrollar sistemas capaces de aprender patrones complejos a partir de grandes cantidades de datos [\[2\]](#). A pesar de estos

avances, el despliegue de vehículos completamente autónomos en entornos urbanos sigue enfrentando severos desafíos tecnológicos, éticos y normativos, derivados de la alta impredecibilidad del tráfico y la necesidad de interactuar de forma segura con la infraestructura existente.

2.2.1.1 Niveles de clasificación de conducción autónoma

- El nivel 0, denominado *sin automatización de conducción* se caracteriza porque el vehículo no realiza ninguna tarea relacionada con la conducción de forma automática. El conductor humano es el responsable de controlar completamente el vehículo, incluyendo la dirección, la aceleración, el frenado y la supervisión del entorno. En este nivel pueden existir sistemas de asistencia como avisos de salida de carril, alertas de colisión o sistemas de frenado automático de emergencia. Sin embargo, estos sistemas únicamente proporcionan información o actúan ante situaciones concretas, sin asumir el control continuo de la conducción.
- El nivel 1, denominado *asistencia al conductor* se caracteriza porque el vehículo comienza a incorporar funciones automatizadas capaces de controlar ciertos aspectos individuales de la conducción. Estos sistemas pueden actuar sobre el control longitudinal, relacionado con la aceleración y el frenado, o sobre el control lateral, encargado de gestionar la dirección del vehículo. A pesar de estas capacidades, el conductor continúa siendo responsable de supervisar el entorno y debe mantener la atención necesaria para intervenir en cualquier momento.
- El nivel 2, conocido como *automatización parcial* corresponde a que permite al vehículo controlar simultáneamente la dirección, la aceleración y el frenado bajo determinadas condiciones. Este nivel incluye muchos de los sistemas avanzados de asistencia a la conducción disponibles actualmente en vehículos comerciales. Aunque el sistema puede mantener la trayectoria y adaptar la velocidad de forma automática, el conductor sigue siendo responsable de supervisar la circulación y debe recuperar el control cuando sea necesario.
- El nivel 3, denominado *automatización condicional*, el sistema pasa a considerarse un sistema de conducción automatizada Automated Driving System (ADS), ya que puede realizar todas las tareas necesarias para conducir dentro de unas condiciones determinadas. En este nivel, el conductor no necesita supervisar continuamente el funcionamiento del vehículo mientras el sistema se encuentre dentro de su dominio operativo permitido. No obstante, debe estar preparado para asumir nuevamente el control cuando el sistema lo solicite debido a que se haya alcanzado una situación que exceda sus capacidades.
- El nivel 4, denominado *alta automatización*, el vehículo es capaz de realizar todas las tareas relacionadas con la conducción dentro de un dominio operacional específico Operational Design Domain (ODD). En este caso, el sistema puede controlar completamente el vehículo y gestionar situaciones inesperadas dentro de los límites para los que ha sido diseñado. A diferencia del nivel anterior, si el sistema alcanza sus restricciones operativas, debe ser capaz de ejecutar una estrategia segura sin depender necesariamente de una intervención inmediata del conductor. Este nivel se encuentra asociado a aplicaciones como vehículos autónomos destinados a operar en zonas concretas o bajo condiciones controladas.
- El nivel 5, denominado *automatización completa*, representa el grado máximo de autonomía definido por la SAE. En este nivel el vehículo es capaz de realizar todas las tareas de conducción en cualquier tipo de carretera y bajo cualquier condición en la que un conductor humano podría operar. El sistema no requiere la presencia de un conductor humano ni supervisión externa, ya que asume completamente la responsabilidad de la conducción.

2.2.2 Simulación para validación vehículos autónomos

El incremento progresivo de los niveles de autonomía implica también un aumento significativo de la complejidad de los sistemas encargados de la percepción, planificación y control del vehículo. A medida que se busca alcanzar los niveles 4 y 5 de la clasificación de la conducción autónoma, el sistema debe estar preparado y ser capaz de tomar decisiones de forma autónoma en entornos altamente dinámicos, interactuando de manera segura con otros vehículos, peatones, infraestructuras y elementos del entorno. Esta complejidad plantea importantes desafíos en los procesos de verificación y validación, ya que resulta necesario garantizar que el vehículo responderá correctamente ante una enorme variedad de situaciones posibles.

Tradicionalmente, la validación de vehículos autónomos se ha basado en pruebas realizadas en entornos reales de circulación. Sin embargo, este enfoque presenta importantes limitaciones cuando se aplica a sistemas de conducción autónoma avanzados. La gran cantidad de escenarios a los que un vehículo autónomo puede estar expuesto durante su funcionamiento hace prácticamente imposible reproducir todas las situaciones relevantes mediante pruebas reales sin comprometer la seguridad del resto de usuarios de la vía. Además, la evaluación de escenarios potencialmente peligrosos o poco frecuentes implica elevados costes económicos y puede introducir riesgos tanto para las personas como para los vehículos involucrados [3].

Ante estas limitaciones, los entornos de simulación se han consolidado como una de las herramientas fundamentales para el desarrollo y validación de sistemas de conducción autónoma. Los entornos simulados permiten recrear condiciones de conducción de forma controlada, segura y repetible, posibilitando la evaluación sistemática del comportamiento del vehículo ante una variedad de escenarios. Gracias a ello, es posible realizar miles de pruebas en un tiempo reducido y sin los riesgos asociados a la experimentación en entornos reales.

Uno de los principales beneficios de la simulación es la capacidad de reproducir situaciones poco frecuentes, pero que en las pocas ocasiones que pueden ser encontradas, resultan especialmente críticas desde el punto de vista de la seguridad vial, tanto para los ocupantes del vehículo autónomo, como para el resto de usuarios que se encuentren en la vía en ese momento, desde peatones a otros vehículos circulando. Escenarios como la aparición repentina de obstáculos, cortes de carretera, obras, comportamientos inesperados de otros usuarios de la vía o condiciones meteorológicas adversas pueden generarse de forma controlada dentro de estos simuladores para analizar la respuesta del sistema ante estas variables del entorno. Este enfoque permite identificar posibles fallos críticos antes de que los algoritmos sean desplegados y probados en vehículos reales[3][4].

En los últimos años, la investigación ha evolucionado desde modelos de validación basados únicamente en la acumulación de kilómetros recorridos hacia metodologías centradas en escenarios específicos. Diversos estudios han demostrado que recorrer grandes distancias, tanto en entornos reales como simulados, no garantiza necesariamente que el sistema haya sido expuesto a las situaciones más críticas para la seguridad. Como consecuencia, ha surgido el paradigma de la validación basada en escenarios, cuyo objetivo consiste en definir y ejecutar conjuntos de situaciones representativas que permitan evaluar de manera sistemática las capacidades del vehículo autónomo [4].

Este enfoque resulta especialmente relevante debido a la enorme cantidad de variaciones presentes durante la conducción. Las posibles combinaciones de tráfico, condiciones ambientales, infraestructura y comportamiento de los diferentes agentes hacen inviable una exploración exhaustiva mediante pruebas convencionales. Por ello, los métodos actuales buscan identificar y generar escenarios críticos capaces de poner a prueba los límites del sistema y revelar comportamientos potencialmente inseguros.

La validación basada en escenarios no se centra únicamente en comprobar si el vehículo es capaz de evitar accidentes o colisiones, sino que también permite analizar otros aspectos relacionados con la seguridad y eficiencia de la conducción [5]. Entre estos aspectos se encuentran el cumplimiento de las normas de circulación, la interpretación correcta de señales y prioridades, la capacidad de mantener distancias adecuadas respecto a otros vehículos y la realización de maniobras eficientes. La simulación facilita este tipo de evaluaciones al proporcionar un entorno controlado donde es posible registrar el comportamiento del vehículo durante la ejecución de diferentes pruebas y analizar posteriormente los resultados obtenidos mediante métricas específicas.

A pesar de las ventajas que ofrece la simulación, existen ciertas limitaciones asociadas a la dificultad de representar con total precisión todas las condiciones del mundo real. Los modelos empleados para representar elementos como los vehículos, sensores, tráfico y condiciones ambientales constituyen aproximaciones de la realidad, por lo que pueden aparecer diferencias entre el comportamiento observado durante las pruebas simuladas y el comportamiento obtenido en entornos reales. Esta diferencia representa uno de los principales retos actuales en la validación de sistemas de conducción autónoma [6].

No obstante, debido a su capacidad para generar escenarios complejos de forma segura, reproducible y económicamente viable, la simulación se ha convertido en un elemento indispensable dentro del ciclo de desarrollo de los sistemas de conducción autónoma. Como consecuencia, durante los últimos años han surgido numerosas plataformas especializadas capaces de recrear entornos urbanos, modelar el comportamiento del tráfico, simular sensores avanzados y proporcionar herramientas para la evaluación de algoritmos de percepción, planificación y control.

Las características, capacidades y aplicaciones de estas plataformas varían significativamente en función de los objetivos para los que fueron diseñadas. Por este motivo, resulta necesario analizar las principales soluciones disponibles en la actualidad con el fin de comprender sus ventajas, limitaciones y ámbitos de aplicación dentro de la investigación en conducción autónoma.

2.2.3 Principales simuladores para conducción autónoma

El uso de simuladores se ha convertido en una práctica fundamental para el desarrollo y validación de sistemas de conducción autónoma, ya que permiten reproducir escenarios complejos de forma segura, controlada y reproducible. Como consecuencia, durante los últimos años han surgido diversas plataformas orientadas a la investigación y evaluación de vehículos autónomos, cada una con características y funcionalidades específicas. En esta sección se presentan algunos de los simuladores más representativos utilizados en este ámbito.

- **AirSim:** AirSim es un simulador de código abierto desarrollado por Microsoft sobre el motor gráfico Unreal Engine. Fue diseñado inicialmente para la simulación de drones, aunque posteriormente incorporó soporte para vehículos terrestres. Proporciona sensores virtuales como cámaras RGB, sensores de profundidad, LiDAR e IMU, siendo especialmente útil para la generación de datos sintéticos destinados al entrenamiento de modelos y evaluación de algoritmos de inteligencia artificial[7].

Su principal ventaja radica en la calidad visual de los escenarios y en su integración con entornos de aprendizaje automático. Sin embargo, su desarrollo ha perdido actividad en los últimos años y sus capacidades específicas para simulación y conducción en entornos urbanos son más limitadas.

- **LGSVL/SVL:** Es una plataforma de código abierto orientada específicamente a la validación de sistemas de conducción autónoma. Está basada en Unity y ofrece integración con sistemas como Apollo y Autoware[8].

Entre sus características destacan la simulación de múltiples sensores, la creación de escenarios complejos y la posibilidad de evaluar algoritmos completos de conducción autónoma. No obstante, presenta una comunidad más reducida y una menor disponibilidad de mapas y recursos en comparación con otras alternativas.

- **Gazebo:** Es uno de los simuladores más empleados en el ámbito de la robótica. Aunque su propósito principal no está orientado específicamente a la conducción autónoma, dispone de paquetes y extensiones que permiten la simulación de vehículos terrestres, sensores y entornos necesarios para la evaluación de sistemas de conducción autónoma. Además, proporciona un motor de simulación física avanzado y una integración nativa con ROS (Robot Operating System), lo que lo convierte en una herramienta ampliamente utilizada en investigación y desarrollo[9].

No obstante, su enfoque continúa estando orientado a la robótica general. Asimismo, la calidad gráfica y el nivel de realismo visual de los entornos urbanos suelen ser inferiores a los ofrecidos por simuladores basados en motores gráficos avanzados, como Unreal Engine.

- **CARLA:** Es un simulador de código abierto desarrollado específicamente para la investigación, entrenamiento y validación de sistemas de conducción autónoma. Está construido sobre Unreal Engine, lo que le permite ofrecer entornos urbanos con un elevado grado de realismo visual y físico. Asimismo, utiliza el estándar OpenDRIVE para la definición de carreteras y escenarios urbanos, facilitando la creación e importación de mapas personalizados[10].

CARLA proporciona una amplia variedad de sensores virtuales, entre los que se incluyen cámaras RGB, cámaras de profundidad, LiDAR, radar, GPS e IMU, permitiendo generar datos sintéticos para el desarrollo y evaluación de algoritmos de percepción y navegación. Además, dispone de una API en Python que permite controlar completamente la simulación, los vehículos, las condiciones meteorológicas y el tráfico presente en el entorno en tiempo real.

Entre sus funcionalidades destacan la gestión automática del tráfico mediante el Traffic Manager, la grabación y reproducción de simulaciones, la integración con ROS y la posibilidad de crear escenarios personalizados mediante mapas y activos abiertos. Además, la compatibilidad de **CARLA** con herramientas de generación y edición de mapas, como RoadRunner, permite incorporar modificaciones específicas en los entornos urbanos, facilitando la creación de escenarios personalizados con bloqueos y obstáculos.

Tabla 2.1: Comparativa entre simuladores.

Característica	AirSim	SVL	Gazebo	CARLA
Código abierto	Sí	Sí	Sí	Sí
Motor gráfico	Unreal Engine	Unity	Propio/Ogre	Unreal Engine
Capacidad de simulación urbana	Medio	Alto	Bajo	Muy alto
Sensores y percepción	Básico	Avanzado	Medio	Muy avanzado
Integración ROS	Parcial	Excelente	Completo	Completo
Datos sintéticos	Sí	Sí	Limitado	Sí
Realismo visual	Alto	Alto	Medio	Muy alto
Modificación mapas	Limitado	Medio	Medio	Alto
Soporte comunidad	Alta	Media	Muy alta	Muy alta

CARLA fue desarrollado específicamente para dar soporte al desarrollo, entrenamiento y validación de sistemas de conducción autónoma en entornos urbanos de alta fidelidad. A diferencia de otros simuladores de propósito general, esta plataforma destaca por ofrecer un renderizado visual fotorrealista, una amplia gama de sensores virtuales configurables y un control dinámico de las condiciones ambientales y de tráfico. Estas capacidades permiten recrear situaciones viales complejas bajo un entorno controlado, lo que garantiza la reproducibilidad de los experimentos y la obtención de métricas de rendimiento objetivas.

Teniendo en cuenta la comparativa expuesta entre las diferentes plataformas del mercado, CARLA se consolida como la herramienta idónea por su capacidad de integración con herramientas externas de edición como RoadRunner. Esta sinergia facilita la importación de geografía real y la posterior generación de escenarios personalizados con distintos tipos de bloqueos en la vía. De este modo, el simulador permite registrar con precisión el comportamiento del vehículo autónomo y extraer secuencias visuales para su posterior análisis semántico mediante LLM, proporcionando un entorno de pruebas robusto para evaluar el sistema ante imprevistos.

2.2.4 Modelos de lenguaje aplicados a la visión

El desarrollo de los LLM ha supuesto uno de los avances más significativos de los últimos años en el ámbito de la inteligencia artificial. Gracias al incremento de la capacidad computacional, al aumento del tamaño de los modelos y a la disponibilidad de grandes volúmenes de datos para su entrenamiento, estos sistemas han adquirido capacidades avanzadas de comprensión, generación y razonamiento sobre información textual. Sin embargo, los LLMs tradicionales presentan una limitación fundamental: únicamente pueden procesar texto y carecen de mecanismos para interpretar información visual.

Como respuesta a esta restricción surgieron los Multimodal Large Language Models (MLLM), que combinan las capacidades de razonamiento propias de los LLM con módulos especializados en el procesamiento de imágenes, vídeo y otras modalidades de información. Esta integración ha dado lugar a una nueva generación de sistemas capaces de comprender escenas visuales, describir su contenido y realizar razonamientos complejos a partir de información multimodal. Entre estos sistemas destacan los Visual-Language Model (VLM), que integran información visual y textual para proporcionar una comprensión más completa del entorno, siendo actualmente empleados en aplicaciones como el análisis de entornos urbanos, sistemas avanzados de asistencia a la conducción y en los sistemas de conducción autónoma.

2.2.4.1 Evolución de los LLM

Los LLM han experimentado una evolución significativa en los últimos años debido principalmente al incremento de la cantidad de datos utilizados durante su entrenamiento, el aumento de la capacidad computacional disponible y el escalado del tamaño de los propios modelos. Este crecimiento ha permitido la aparición de capacidades avanzadas como el seguimiento de instrucciones, el aprendizaje en contexto y el razonamiento mediante cadenas de pensamiento o Chain of Thought. Gracias a estas mejoras, los LLM tradicionales han alcanzado resultados sobresalientes en tareas relacionadas con el procesamiento del lenguaje natural, generación de texto y resolución de problemas complejos de razonamiento[11][12].

Sin embargo, los LLM tradicionales presentan una limitación fundamental y es que están diseñados principalmente para procesar información textual y no poseen una capacidad para percibir e interpretar información procedente del mundo físico. Por este motivo, no son capaces de analizar directamente información visual como imágenes o vídeos, identificar elementos presentes en una escena o comprender relaciones espaciales entre diferentes objetos sin la incorporación de mecanismos adicionales[13].

Paralelamente al desarrollo de los [LLM](#), los modelos especializados en visión artificial han evolucionado con el objetivo de extraer información relevante a partir de imágenes y vídeos. Estos modelos son capaces de realizar tareas como detección de objetos, segmentación semántica, clasificación de imágenes o generación de descripciones visuales. No obstante, aunque poseen una gran capacidad de percepción visual, tradicionalmente presentan limitaciones en tareas que requieren razonamiento complejo, comprensión contextual o integración de información de diferentes modalidades.

Como respuesta a estas limitaciones surgen los [MLLM](#), cuyo objetivo es combinar las capacidades de percepción de los modelos visuales con las capacidades de comprensión y razonamiento de los [LLM](#). Estos modelos integran un modelo de lenguaje con componentes especializados capaces de procesar diferentes tipos de información, como imágenes, vídeo o audio, permitiendo realizar tareas que requieren la combinación de varias modalidades.

Los [MLLM](#) han permitido desarrollar nuevas capacidades difíciles de alcanzar mediante métodos tradicionales, como la descripción automática de escenas, la respuesta a preguntas relacionadas con contenido visual, la generación de contenido multimodal o la resolución de problemas donde es necesario relacionar información visual y textual. Dentro de esta evolución aparecen los [VLM](#), diseñados específicamente para establecer una relación entre representaciones visuales y lenguaje natural. Estos modelos son capaces de analizar imágenes, generar descripciones detalladas del contenido observado y responder preguntas relacionadas con el entorno visual[14].

La aparición de modelos como GPT-4V supuso un punto de inflexión dentro de la investigación en inteligencia artificial multimodal, demostrando la capacidad de los modelos de lenguaje avanzados para integrar información visual y realizar tareas de comprensión y razonamiento sobre imágenes. A partir de este avance, tanto la industria como la comunidad científica comenzaron a desarrollar nuevos modelos capaces de procesar múltiples modalidades, ampliando progresivamente sus capacidades de percepción y comprensión del entorno[15].

Actualmente existe una amplia variedad de modelos multimodales desarrollados tanto por grandes grupos de investigación como por la comunidad de inteligencia artificial de código abierto. Estos modelos presentan diferentes arquitecturas, tamaños y especializaciones, permitiendo adaptarlos a distintos escenarios de aplicación, desde tareas generales de comprensión multimodal hasta dominios específicos como medicina, robótica o conducción autónoma.

la capacidad de generalización de los modelos, seguía siendo necesario disponer de datos etiquetados para cada nueva tarea. La búsqueda de métodos más flexibles y escalables impulsó el desarrollo de los modelos multimodales y, en particular, de los denominados **VLM**[17].

Los **VLM** constituyen una nueva generación de modelos capaces de procesar simultáneamente información visual y textual. A diferencia de los modelos tradicionales de visión por computador, que únicamente analizan imágenes, los **VLM** aprenden relaciones semánticas entre imágenes y descripciones a partir de grandes colecciones de pares imagen-texto. Gracias a este proceso de aprendizaje, el modelo desarrolla una representación conjunta de ambas modalidades, permitiendo comprender conceptos visuales mediante información textual y viceversa.

Una de las características más importantes de estos modelos es su capacidad para realizar predicciones en escenarios no observados previamente durante el entrenamiento. Este paradigma permite que un modelo identifique objetos o conceptos utilizando únicamente descripciones textuales, sin necesidad de haber sido entrenado específicamente para cada categoría. En lugar de aprender clasificadores independientes para cada tarea, el modelo compara la similitud entre la representación visual de una imagen y la representación textual de una descripción, seleccionando aquella que presente una mayor correspondencia semántica. Esta capacidad supone un avance significativo frente a los métodos tradicionales, ya que reduce la necesidad de recopilar y etiquetar grandes cantidades de datos específicos para cada aplicación.

El desarrollo moderno de los **VLM** está estrechamente relacionado con la aparición del modelo Contrastive Language-Image Pretraining (CLIP), presentado por OpenAI en 2021. Este trabajo demostró que era posible entrenar modelos capaces de relacionar imágenes y lenguaje natural utilizando cientos de millones de pares imagen-texto. **CLIP** emplea una estrategia de aprendizaje contrastivo mediante la cual las imágenes y sus descripciones asociadas se proyectan a un mismo espacio semántico. Durante el entrenamiento, el modelo aprende a aproximar las representaciones de las imágenes y textos relacionados mientras aleja aquellas que no guardan relación entre sí. Los resultados obtenidos mostraron una capacidad de generalización sin precedentes y marcaron el inicio de una nueva línea de investigación centrada en los modelos multimodales[17].

En cuanto a la composición en la arquitectura de los **VLM**, suele dividirse en dos módulos principales:

- El primero consiste en un codificador visual que se encarga de extraer características relevantes de las imágenes; para esta tarea se emplean habitualmente arquitecturas basadas en redes convolucionales profundas o, más recientemente, modelos basados en transformadores visuales.
- El segundo módulo que conforma los **VLM** es el codificador textual, generalmente construido a partir de arquitecturas Transformer, similares a los **LLM** tradicionales como BERT o GPT.

Gracias a las capacidades de los **VLM**, estos han demostrado un rendimiento destacado en una amplia variedad de tareas de visión por computador, tales como la clasificación de imágenes, la detección de objetos, la segmentación semántica y el reconocimiento de acciones en secuencias de vídeo. La posibilidad de integrar información visual y textual dentro de una misma arquitectura convierte a estos modelos en herramientas especialmente adecuadas para escenarios complejos donde es necesario comprender el contexto de una escena y generar interpretaciones semánticas de alto nivel[17].

Las capacidades de razonamiento multimodal proporcionadas por los **VLM** han favorecido su adopción en numerosos ámbitos de investigación, entre los que destaca la conducción autónoma. En este dominio, la literatura reciente ha comenzado a explorar el uso de estos modelos para interpretar escenas de tráfico complejas, identificar señales, detectar obstáculos imprevistos y, fundamentalmente, generar explicaciones

semánticas en lenguaje natural sobre el comportamiento del entorno vial. Al procesar secuencias de vídeo en lugar de imágenes estáticas, arquitecturas avanzadas y modelos de lenguaje visual adaptados a vídeo permiten realizar un análisis cronológico y contextual de las rutas viales. Como consecuencia, los **VLM** representan una tecnología emergente y prometedora para complementar los sistemas tradicionales de percepción, dotando al vehículo simulado de una capacidad de razonamiento semántico superior ante bloqueos u anomalías en la infraestructura.

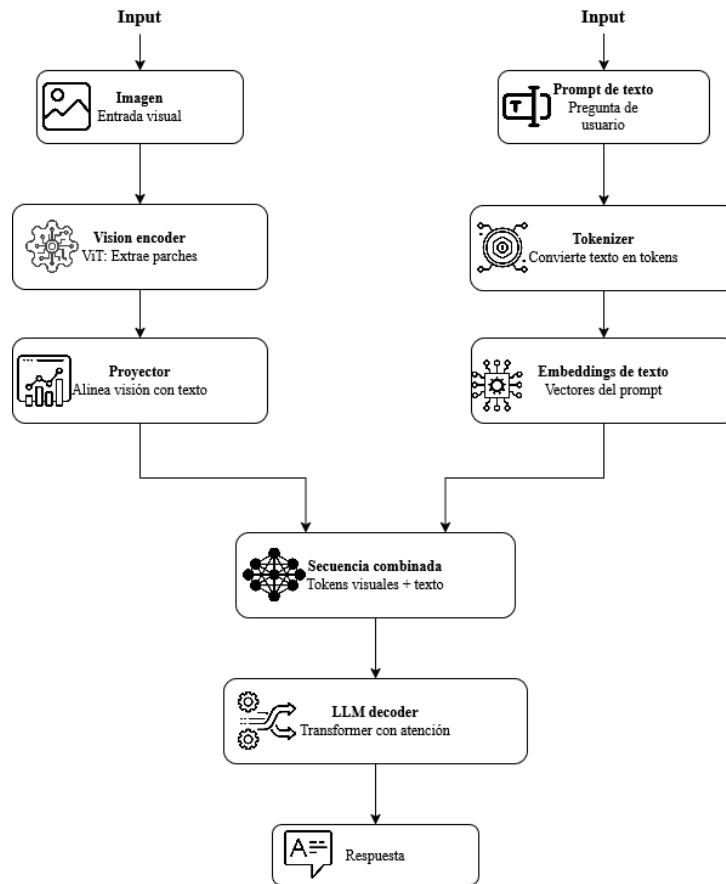


Figura 2.2: Flujo de un **Visual-Language Model (VLM)**

2.2.4.3 Aplicación de los modelos de lenguaje visual a la conducción autónoma

Tradicionalmente, los sistemas de conducción autónoma se han basado principalmente en arquitecturas modulares, donde cada una de las tareas principales, entre las que destacan la percepción, predicción, planificación y control, se realizaba mediante módulos independientes. Aunque estos sistemas han alcanzado un elevado rendimiento en entornos controlados, pueden presentar dificultades cuando deben interpretar situaciones complejas o poco frecuentes debido a su limitada capacidad de razonamiento semántico. En este contexto, los **VLM** surgen como una alternativa capaz de integrar información visual y lingüística para proporcionar una comprensión más completa del entorno de conducción.

Los **VLM** combinan modelos de visión por computador con **LLM**, permitiendo transformar información visual procedente de sensores, como cámaras, en representaciones semánticas expresadas mediante lenguaje natural. Esta capacidad permite no solo detectar elementos presentes en el entorno, sino también establecer relaciones entre objetos, describir situaciones y realizar inferencias basadas en el contexto de la escena observada. A continuación, se describen algunas de las principales aplicaciones de los **VLM** en sistemas de conducción autónoma, tanto en entornos reales como simulados[18].

- **Percepción y comprensión del entorno:** Una de las principales aplicaciones de los **VLM** en conducción autónoma es la mejora de la percepción y comprensión del entorno. Mientras que los modelos tradicionales de visión artificial suelen centrarse en tareas específicas, como la detección de vehículos, peatones o señales de tráfico, los **VLM** permiten realizar una interpretación más generalizada de la escena en la que se encuentra el vehículo.

Por ejemplo, un **VLM** puede analizar una imagen o una secuencia de imágenes y generar una descripción semántica como: "Existe un obstáculo situado en la trayectoria del vehículo que bloquea parcialmente el carril actual, por lo que es necesario modificar la ruta prevista".

Este tipo de representación resulta especialmente relevante debido a que introduce un nivel de comprensión más cercano al razonamiento humano, permitiendo identificar no solamente los objetos presentes en la escena, sino también su importancia y relación dentro del contexto de conducción.

- **Navegación y planificación:** Otra aplicación relevante de los **VLM** es su integración en tareas de navegación y planificación. Los modelos de lenguaje aportan conocimiento semántico que puede complementar la información geométrica obtenida mediante sensores y sistemas tradicionales de percepción.

En lugar de limitarse únicamente a seguir reglas predefinidas, un sistema basado en **VLM** puede interpretar instrucciones de alto nivel y relacionarlas con el entorno observado. De esta forma, permite avanzar hacia sistemas donde la planificación no depende exclusivamente de mapas estáticos o reglas programadas, sino también de una interpretación contextual del escenario de conducción.

Esta capacidad resulta especialmente interesante en escenarios dinámicos, donde pueden aparecer situaciones inesperadas como obstáculos temporales, modificaciones de la infraestructura o cambios en las condiciones del tráfico.

- **Toma de decisiones:** Una ventaja destacada de los **VLM** es su capacidad para generar explicaciones en lenguaje natural. En conducción autónoma, donde la seguridad es un aspecto crítico, comprender la razón detrás de una determinada decisión resulta tan importante como la propia acción ejecutada por el vehículo.

Los **VLM** pueden actuar como una capa adicional de razonamiento capaz de justificar decisiones tomadas por el sistema autónomo, proporcionando explicaciones como la identificación de un obstáculo, la necesidad de cambiar de trayectoria o la adaptación de la velocidad ante una situación concreta.

Esta capacidad puede mejorar la interpretabilidad de los sistemas autónomos y facilitar procesos de validación, depuración y análisis del comportamiento del vehículo.

- **Generación de datos:** Además de las aplicaciones relacionadas con percepción y planificación, los **VLM** también pueden emplearse para la generación y análisis de información semántica sobre datos de conducción. Los conjuntos de datos utilizados en conducción autónoma requieren grandes cantidades de información etiquetada, cuyo proceso de generación suele ser costoso y requiere intervención humana.

Los **VLM** pueden contribuir a automatizar tareas como la descripción de escenas, generación de etiquetas semánticas o identificación de situaciones específicas dentro de secuencias de conducción.

Esta capacidad resulta especialmente relevante en entornos virtuales como **CARLA**, donde es posible generar múltiples escenarios con diferentes configuraciones de tráfico, infraestructura y obstáculos. De esta forma, los **VLM** pueden utilizarse como herramientas de análisis para evaluar la capacidad de comprensión semántica de los sistemas autónomos en escenarios controlados y reproducibles.

Capítulo 3

Metodología y herramientas empleadas

3.1 Introducción

En este capítulo se presentan las principales herramientas, plataformas y modelos utilizados durante el desarrollo del proyecto. Se describen sus características, funcionalidades y las versiones empleadas, indicando el papel que desempeña cada una dentro del flujo de trabajo seguido. De este modo, el contenido se estructura en torno a las fuentes de datos geográficos, las herramientas de edición y diseño para la modificación de entornos urbanos, el simulador de conducción autónoma CARLA y, finalmente, los modelos de lenguaje multimodales seleccionados para el posterior análisis semántico de la información visual.

3.2 Simulador CARLA

[Car Learning to Act \(CARLA\)](#) es un simulador de conducción autónoma de código abierto desarrollado inicialmente por el Computer Vision Center de la Universidad Autónoma de Barcelona junto con la colaboración de investigadores de Intel. El simulador fue diseñado con una arquitectura modular y flexible con el objetivo de proporcionar un entorno de pruebas donde sea posible desarrollar, entrenar y evaluar diferentes algoritmos relacionados con la conducción autónoma.

Una de las principales ventajas de [CARLA](#) es que permite realizar experimentos en entornos virtuales controlados, reduciendo la necesidad de utilizar vehículos reales durante las primeras fases de desarrollo. Esto permite estudiar el comportamiento de los sistemas autónomos ante diferentes situaciones de circulación, incluyendo escenarios complejos o situaciones potencialmente peligrosas, sin poner en riesgo la seguridad de usuarios reales.

El simulador está orientado a cubrir diferentes áreas dentro del campo de la conducción autónoma, como el desarrollo de sistemas de percepción, la planificación de trayectorias, la toma de decisiones del vehículo o el estudio del comportamiento frente a diferentes condiciones del entorno. Para ello, [CARLA](#) proporciona diferentes herramientas que permiten crear escenarios urbanos, controlar vehículos y obtener información del entorno mediante sensores simulados.

Para la generación y representación de los escenarios, [CARLA](#) utiliza Unreal Engine como motor gráfico, permitiendo crear entornos tridimensionales realistas con elementos urbanos y condiciones

ambientales configurables. Además, emplea el estándar ASAM OpenDRIVE para la definición de la infraestructura vial, permitiendo representar elementos como carreteras, carriles, intersecciones y señalización.

El control de las simulaciones se realiza principalmente mediante una API disponible para lenguajes como Python y C++, desde la cual es posible gestionar los vehículos, modificar parámetros del entorno, añadir sensores y recopilar los datos generados durante la ejecución de las pruebas. Esta característica convierte a **CARLA** en una herramienta adecuada para la creación de escenarios personalizados y la evaluación de sistemas orientados a la conducción autónoma.

3.2.1 Arquitectura del simulador

El simulador **CARLA** se basa en una arquitectura cliente-servidor escalable, donde la simulación se divide entre un proceso encargado de ejecutar el mundo virtual y uno o varios clientes externos que interactúan con dicho entorno. Esta separación permite que el usuario pueda controlar la simulación sin necesidad de modificar directamente el núcleo interno del simulador, facilitando la creación de escenarios personalizados y la automatización de pruebas.

El servidor de **CARLA** es el encargado de ejecutar la simulación y mantener el estado del mundo virtual. Entre sus funciones principales se encuentran la representación gráfica del entorno, la ejecución de las físicas de los vehículos y elementos dinámicos, la actualización del estado de los actores presentes en la escena y la generación de información procedente de los sensores simulados.

Por otro lado, el cliente actúa como elemento de control de la simulación. Este se comunica con el servidor mediante la API proporcionada por **CARLA**, disponible para lenguajes como Python y C++. A través de esta interfaz, el usuario puede crear y modificar elementos del escenario, gestionar vehículos y peatones, establecer condiciones meteorológicas, cambiar parámetros de la simulación y obtener los datos generados durante la ejecución.

La comunicación entre ambos componentes permite que los scripts desarrollados por el usuario o incluidos por defecto en **CARLA** envíen órdenes al servidor, que posteriormente actualiza el entorno simulado en función de los cambios solicitados. Esta arquitectura resulta especialmente útil para tareas relacionadas con conducción autónoma, ya que permite ejecutar experimentos repetibles bajo unas condiciones controladas.

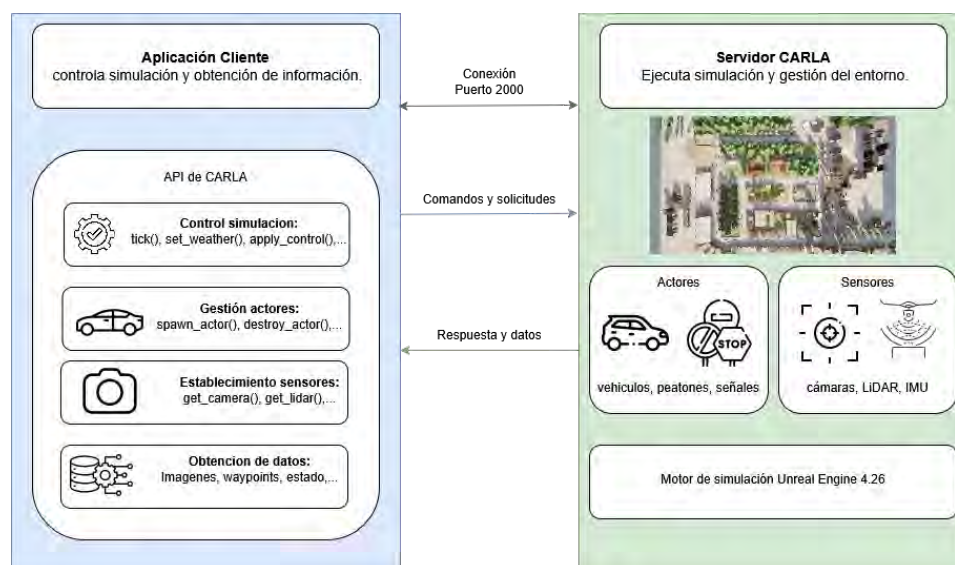


Figura 3.1: Arquitectura Cliente-Servidor de CARLA

3.2.2 Motor gráfico: Unreal Engine

Unreal Engine es un motor gráfico y de simulación tridimensional desarrollado por Epic Games. Aunque inicialmente fue concebido para el desarrollo de videojuegos, actualmente se emplea en numerosos ámbitos como la producción audiovisual, la visualización arquitectónica, el diseño industrial y la simulación de sistemas complejos. Su capacidad para generar entornos virtuales detallados y ejecutar simulaciones en tiempo real ha favorecido su adopción en aplicaciones de investigación relacionadas con la robótica y la conducción autónoma.

CARLA se encuentra desarrollado sobre Unreal Engine debido a las capacidades que este proporciona para la representación de entornos urbanos realistas. Uno de los principales retos en el desarrollo de sistemas de conducción autónoma consiste en reducir la diferencia existente entre los escenarios simulados y las condiciones presentes en el mundo real. Cuanto mayor sea el nivel de realismo de la simulación, más útiles resultarán los datos generados para el entrenamiento y evaluación de algoritmos de percepción y toma de decisiones.

Para abordar este problema, Unreal Engine proporciona un sistema de renderizado avanzado capaz de representar materiales, iluminación, sombras y condiciones ambientales de forma realista. Gracias a ello, los sensores simulados de CARLA, especialmente las cámaras RGB, pueden generar imágenes con un elevado nivel de detalle visual, permitiendo aproximar las condiciones que encontraría un vehículo autónomo durante su operación en entornos reales.

Además de las capacidades gráficas, Unreal Engine incorpora un sistema de físicas que permite simular de manera realista el comportamiento de vehículos y otros elementos dinámicos presentes en la escena. Este sistema contempla aspectos como colisiones, gravedad, interacción con objetos del entorno y comportamiento dinámico de los vehículos, contribuyendo a aumentar el realismo de las pruebas realizadas dentro del simulador.

Otra característica relevante es la posibilidad de integrar diferentes tipos de sensores virtuales. CARLA aprovecha las funcionalidades del motor para generar cámaras, sensores de profundidad, radares y sensores LiDAR, proporcionando información que posteriormente puede ser utilizada para el desarrollo y evaluación de algoritmos de conducción autónoma.

Por último, Unreal Engine dispone de un entorno de edición visual que facilita la creación y modificación de escenarios tridimensionales. Esta capacidad resulta especialmente útil para la generación de mapas personalizados, la incorporación de elementos urbanos y la configuración de diferentes condiciones de simulación empleadas durante el desarrollo del presente trabajo.

Actualmente, CARLA dispone de versiones basadas en diferentes ediciones de Unreal Engine. La rama más consolidada del simulador se encuentra desarrollada sobre Unreal Engine 4.26, utilizada desde la versión 0.9.12 de CARLA y ampliamente empleada en entornos de investigación y desarrollo.



Figura 3.2: Ejemplo de la interfaz base de Unreal Engine y la simulación de CARLA ejecutándose sobre este.

3.2.3 ASAM OpenDRIVE

ASAM OpenDRIVE es un estándar abierto utilizado para la descripción de redes viales en entornos de simulación. Basado en un formato estructurado en XML, este estándar representa la lógica y geometría de una infraestructura vial mediante información matemática y descriptiva, independientemente de la representación gráfica utilizada.

A diferencia de un modelo tridimensional generado por un motor gráfico, OpenDRIVE no almacena directamente elementos visuales como texturas o modelos 3D. En su lugar, define la estructura de las carreteras, lo que incluye la geometría de las vías, la distribución de carriles, las conexiones entre segmentos, las limitaciones de velocidad, las marcas viales y otros elementos necesarios para la interpretación correcta de la infraestructura.

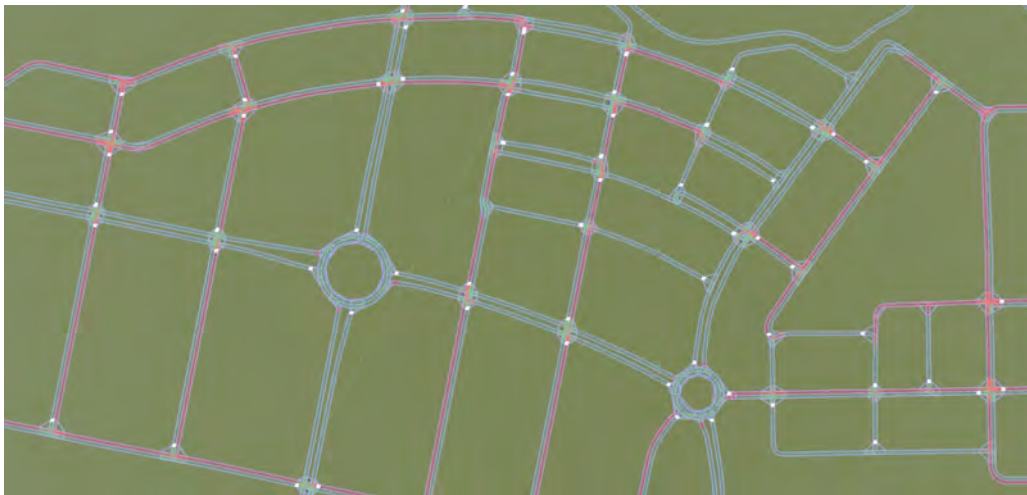


Figura 3.3: Definición de carriles contenido en archivo .xodr

Dentro del simulador [CARLA](#), OpenDRIVE desempeña un papel fundamental al separar la representación visual del entorno de la lógica de circulación. Mientras que Unreal Engine genera la apariencia tridimensional del escenario, OpenDRIVE proporciona la información geométrica necesaria para que el simulador reconozca la estructura de las carreteras y precise la navegación de los vehículos dentro del entorno[10].

A partir de los archivos de este estándar, CARLA genera una representación lógica de la red vial mediante elementos como waypoints, los cuales definen trayectorias válidas para los vehículos y agilizan el funcionamiento de sistemas internos como el Traffic Manager. Esta estructura permite la creación de rutas, mantiene los vehículos dentro de sus carriles correspondientes y gestiona los movimientos complejos en intersecciones.

Además, el uso de este estándar facilita la interoperabilidad entre las diversas herramientas de creación de escenarios. Por ejemplo, plataformas de diseño y edición de carreteras como RoadRunner generan mapas compatibles con CARLA mediante archivos OpenDRIVE, lo que habilita la transferencia de la información vial sin necesidad de reconstruir manualmente la lógica del escenario.

3.2.4 Mapas del simulador

Para la realización de simulaciones, CARLA proporciona una serie de escenarios urbanos y rurales predefinidos denominados *Towns*. Estos mapas permiten evaluar diferentes aspectos relacionados con la conducción autónoma, como la percepción del entorno, la planificación de trayectorias y el control del vehículo en diferentes condiciones de circulación.

Cada escenario está compuesto por una representación tridimensional del entorno utilizada para la visualización y simulación, junto con una descripción estructurada de la infraestructura vial mediante archivos compatibles con el estándar ASAM OpenDRIVE. Esta separación permite que el simulador gestione tanto la representación gráfica del entorno como la lógica necesaria para la circulación de los vehículos.

La documentación oficial de CARLA incluye diferentes mapas con características específicas. En la Tabla 3.1 se resumen los principales escenarios disponibles y su utilidad dentro del contexto de simulación.

Tabla 3.1: Mapas incluidos en CARLA y características generales.

Mapa	Tipo	Características principales
Town01	Urbano básico	Entorno con estructura regular e intersecciones simples. Adecuado para pruebas iniciales de circulación y control del vehículo.
Town02	Urbano residencial	Mapa de menor tamaño y complejidad utilizado para pruebas rápidas y validaciones iniciales.
Town03	Urbano complejo	Incluye rotondas, túneles, intersecciones y diferentes configuraciones viales que permiten evaluar navegación más compleja.
Town04	Mixto	Combina zonas urbanas con vías de mayor velocidad para estudiar diferentes condiciones de conducción.
Town05	Urbano denso	Escenario con mayor densidad de tráfico e interacciones entre vehículos.
Town06	Autopista	Diseñado para pruebas relacionadas con incorporación, salidas y cambios de carril.
Town07	Rural	Incluye carreteras estrechas y curvas, orientado a escenarios alejados de entornos urbanos.
Town10 HD	Alta definición	Entorno urbano con mayor nivel de detalle visual y complejidad gráfica.
Town11-Town15	Mapas de gran escala	Escenarios de mayor extensión destinados a simulaciones donde se requieren entornos urbanos y rurales de mayor tamaño.

Cabe destacar que las versiones de [CARLA](#) incluyen también los mapas Town08 y Town09. Sin embargo, estos escenarios no se consideran dentro de esta clasificación debido a que, según la documentación oficial del simulador, están principalmente asociados a evaluaciones y desafíos específicos desarrollados por la comunidad.



(a) Town02 desde ejecución de simulación



(b) Vista aérea de Town02

Figura 3.4: Vistas de diferentes ángulos de mapa predeterminado Town02



(a) Town05 desde ejecución de simulación



(b) Vista aérea de Town05

Figura 3.5: Vistas de diferentes ángulos de mapa predeterminado Town05.



(a) Town06 desde ejecución de simulación



(b) Vista aérea de Town06

Figura 3.6: Vistas de diferentes ángulos de mapa predeterminado Town06.



(a) Town10 desde ejecución de simulación



(b) Vista aérea de Town10

Figura 3.7: Vistas de diferentes ángulos de mapa predeterminado Town10.

3.2.5 Sistema de Sensores

Para la realización de simulaciones, [CARLA](#) permite incorporar diferentes tipos de sensores virtuales que pueden asociarse a los vehículos u otros actores presentes en el entorno. Estos sensores permiten obtener información del escenario simulado y reproducir, de forma controlada, el comportamiento de los dispositivos empleados en sistemas reales de conducción autónoma.

Los sensores disponibles pueden clasificarse en función del tipo de información proporcionada. Por un lado, se encuentran los sensores destinados a la percepción del entorno, los cuales intentan simular las limitaciones y características de los sensores utilizados en vehículos reales. Por otro lado, [CARLA](#) proporciona sensores de referencia o **Ground Truth**, que ofrecen información exacta del estado del entorno simulado y se utilizan principalmente para evaluación y validación de algoritmos.

Los principales sensores utilizados dentro del simulador se resumen en la Tabla [3.2](#).

Tabla 3.2: Principales sensores disponibles en CARLA y clasificación según su función.

Sensor	Tipo	Descripción
Cámara RGB	Percepción pasiva	Captura imágenes del entorno simulado. Es utilizada para tareas de visión artificial y permite obtener información visual similar a la proporcionada por una cámara convencional.
Cámara de segmentación semántica	Percepción pasiva / Ground Truth	Asigna una etiqueta a cada píxel de la imagen según la clase del elemento representado, como vehículos, peatones, carreteras o señales. Facilita la generación de datos etiquetados.
Cámara de profundidad	Percepción pasiva	Proporciona información sobre la distancia entre la cámara y los objetos presentes en la escena, permitiendo obtener una representación de profundidad del entorno.
LiDAR	Percepción activa	Genera una nube de puntos tridimensional mediante la emisión de rayos simulados. Permite representar la geometría del entorno y detectar obstáculos.
RADAR	Percepción activa	Simula un sensor basado en ondas electromagnéticas capaz de estimar distancia y velocidad relativa de los objetos detectados.
IMU	Localización	Proporciona información relacionada con el movimiento del vehículo, como aceleración y velocidad angular.
GNSS	Localización	Entrega la posición del vehículo mediante coordenadas geográficas simuladas.
Detector de colisiones	Ground Truth	Detecta impactos entre actores del entorno y proporciona información sobre el evento ocurrido.
Detector de invasión de carril	Ground Truth	Indica si el vehículo abandona el carril asignado durante la simulación.

3.2.6 API de Python

La interacción con el simulador [CARLA](#) se realiza mediante una arquitectura cliente-servidor en la que el servidor ejecuta la simulación física y gráfica, mientras que uno o varios clientes externos pueden controlar los diferentes elementos presentes en el entorno. Para implementar esta comunicación, [CARLA](#) proporciona [API](#) tanto en C++ como en Python. No obstante, la [API](#) de Python constituye la opción más utilizada dentro de la comunidad de usuarios debido a su facilidad de uso, la disponibilidad de ejemplos oficiales y la rápida incorporación de nuevas funcionalidades en las distintas versiones del simulador.

La [API](#) de Python permite acceder y controlar prácticamente todos los componentes de la simulación, incluyendo la creación y eliminación de actores, la configuración de sensores, la gestión del tráfico, la definición de rutas y la obtención de datos procedentes del entorno virtual. Gracias a esta interfaz es posible automatizar experimentos completos sin necesidad de interactuar manualmente con el simulador, lo que resulta especialmente útil en entornos de investigación donde se requiere ejecutar múltiples pruebas bajo condiciones controladas y reproducibles.

Uno de los conceptos fundamentales de la [API](#) es la clase *Actor*, que representa cualquier entidad existente dentro de la simulación. Vehículos, peatones, sensores, señales o elementos estáticos son tratados internamente como actores. Cada actor dispone de una serie de atributos y métodos que permiten consultar su estado o modificar su comportamiento durante la ejecución de la simulación.

La creación de actores se realiza mediante el uso de la clase *BlueprintLibrary*, la cual almacena las definiciones de todos los objetos disponibles en [CARLA](#). Cada definición, denominada *blueprint*, contiene la información necesaria para generar una instancia concreta de un vehículo, un peatón o un sensor. A partir de estos *blueprints*, la función *spawn actor* permite insertar nuevos elementos en posiciones específicas del escenario.

La ubicación y orientación de los actores se define mediante la clase *Transform*, compuesta por una posición tridimensional y una rotación espacial. Esta estructura resulta esencial para situar vehículos y sensores en puntos concretos del mapa o para modificar dinámicamente su localización durante la simulación.

Otro componente relevante es la clase *Waypoint*, que proporciona acceso a la red viaria descrita en el mapa mediante el estándar ASAM OpenDRIVE. Los waypoints permiten obtener información sobre carriles, intersecciones, límites de velocidad y conexiones entre segmentos de carretera, facilitando la planificación de trayectorias y la definición de rutas de circulación para los vehículos.

En lo referente a la adquisición de información, la [API](#) incorpora una amplia variedad de sensores virtuales agrupados bajo la clase *Sensor*. Entre ellos se incluyen los que se pueden observar en la [tabla 3.2](#). Cada sensor puede configurarse individualmente y generar datos sincronizados con la simulación, permitiendo la captura de imágenes, vídeos y mediciones necesarias para el desarrollo de sistemas de percepción y análisis.

La gestión automática de vehículos puede realizarse mediante el módulo *Traffic Manager*, encargado de coordinar el comportamiento del tráfico dentro del entorno virtual. Este componente permite controlar aspectos como la velocidad de circulación, el mantenimiento de distancias de seguridad, los cambios de carril o la interacción con otros vehículos presentes en la escena.

Adicionalmente, la clase *World* y su configuración asociada *WorldSettings* proporcionan mecanismos para controlar parámetros globales de la simulación, tales como el modo de sincronización, la frecuencia de actualización, las condiciones meteorológicas o la gestión temporal de los distintos actores presentes en el escenario.

Tabla 3.3: Principales clases y módulos de la API de Python de CARLA utilizados en el desarrollo del proyecto.

Clase/Módulo	Categoría	Descripción
Actor	Gestión de entidades	Clase base que representa cualquier elemento presente en la simulación, incluyendo vehículos, peatones, sensores y objetos del entorno. Permite consultar y modificar propiedades asociadas a cada entidad.
BlueprintLibrary	Generación de actores	Contiene el catálogo de <i>blueprints</i> disponibles en CARLA. Estos definen las características necesarias para crear vehículos, sensores y otros elementos del entorno simulado.
SpawnActor	Generación de actores	Permite instanciar nuevos actores dentro de la simulación a partir de un <i>blueprint</i> y una posición determinada. Es utilizado para añadir vehículos, sensores u otros objetos al escenario.
Transform	Posicionamiento espacial	Define la posición y orientación de un actor mediante coordenadas tridimensionales y ángulos de rotación. Resulta fundamental para ubicar correctamente los elementos dentro del mapa.
Waypoint	Navegación y rutas	Proporciona acceso a la red viaria descrita mediante OpenDRIVE. Permite obtener información sobre carriles, intersecciones y conexiones entre segmentos de carretera para la planificación de trayectorias.
Sensor	Adquisición de datos	Clase base para los distintos sensores disponibles en CARLA. Permite capturar información visual y geométrica del entorno, como imágenes RGB, datos LiDAR o información de localización.
Traffic Manager	Control de tráfico	Sistema encargado de gestionar el comportamiento automático de los vehículos, incluyendo velocidad, seguimiento de carril, cambios de carril e interacción con otros actores.
World	Gestión del entorno	Representa el mundo simulado y proporciona acceso a los actores, mapas, condiciones meteorológicas y demás elementos que forman parte de la simulación.
WorldSettings	Configuración global	Permite configurar parámetros generales de la simulación, como el modo síncrono, la frecuencia de actualización o aspectos relacionados con la ejecución temporal del entorno.

3.3 OpenStreetMap

[Open Street Map \(OSM\)](#) es un proyecto colaborativo de cartografía digital cuyo objetivo es construir una base de datos geoespacial libre y editable a escala mundial. A diferencia de otros servicios cartográficos comerciales, OpenStreetMap pone a disposición de los usuarios información geográfica bajo una licencia

abierta, permitiendo su utilización, modificación y distribución tanto en proyectos de investigación como en aplicaciones comerciales, siempre que se respeten las condiciones de atribución establecidas por la licencia Open Database License.

La información almacenada en OpenStreetMap es mantenida de forma colaborativa por una comunidad internacional de voluntarios, organismos públicos e instituciones que contribuyen mediante la incorporación y actualización continua de datos geográficos. Estos datos describen con gran nivel de detalle elementos presentes en el entorno real, tales como carreteras, intersecciones, carriles, zonas peatonales, señales, infraestructuras ferroviarias, áreas verdes y numerosos puntos de interés. Toda esta información se organiza mediante una estructura basada en **nodes**, **ways** y **relations**, que permiten representar tanto elementos puntuales como estructuras geográficas de mayor complejidad.

Una de las principales ventajas de OpenStreetMap es la posibilidad de exportar la información geográfica correspondiente a una región determinada en formato *.osm*. Este formato almacena la geometría y los atributos asociados a los distintos elementos cartográficos, constituyendo una representación estructurada del entorno urbano que puede ser procesada posteriormente por diferentes herramientas de modelado y simulación.

En el ámbito de la simulación de vehículos autónomos, OpenStreetMap puede funcionar como una fuente de información para la generación de escenarios realistas. Diversas herramientas, entre ellas RoadRunner, han incorporado herramientas para importar directamente archivos *.osm* para reconstruir la red viaria de una localización real y utilizarla como punto de partida para el diseño de nuevos escenarios. Posteriormente, dichos entornos pueden enriquecerse mediante la incorporación de señalización, mobiliario urbano, edificios u otros elementos tridimensionales antes de ser exportados a simuladores como [CARLA](#).

OpenStreetMap se empleó como fuente de datos geográficos para la creación de escenarios urbanos basados en ubicaciones reales. A partir de la exportación de áreas específicas en formato *.osm*, fue posible importar la información cartográfica en RoadRunner, donde se realizaron las modificaciones necesarias para adaptar los mapas a los experimentos desarrollados. Entre estas modificaciones se incluyó la incorporación de distintos tipos de bloqueos en la red viaria, con el objetivo de generar escenarios controlados que posteriormente fueron utilizados en [CARLA](#) para la ejecución de las rutas de prueba y la obtención de los vídeos empleados en el análisis semántico mediante modelos de lenguaje.

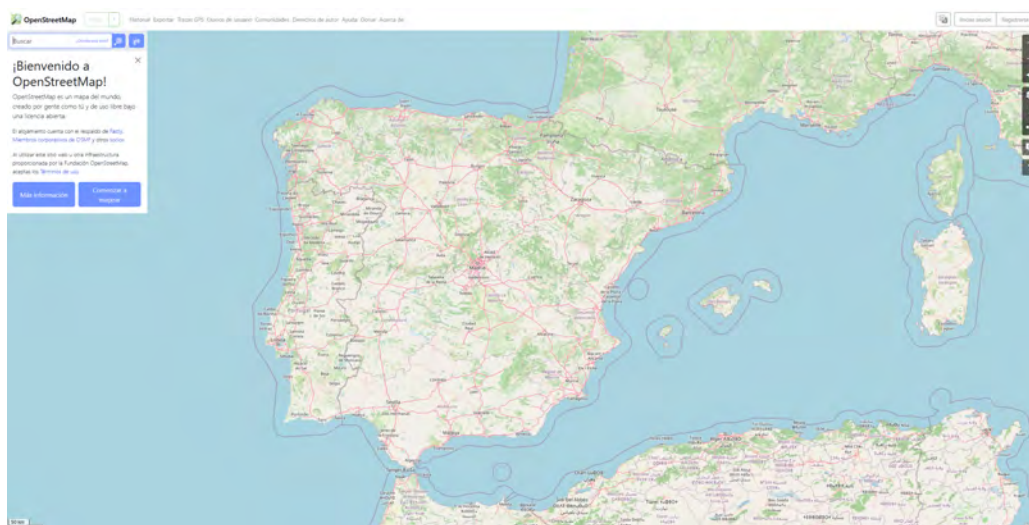


Figura 3.8: Interfaz de selección de entorno urbano OpenStreetMap

3.4 RoadRunner

RoadRunner es un entorno de desarrollo especializado para el diseño y edición de escenarios destinados a simuladores de conducción y sistemas de vehículos autónomos. Desarrollado por MathWorks, esta herramienta permite construir entornos urbanos altamente detallados mediante la creación o modificación de redes viarias, facilitando la generación de escenarios realistas para tareas de simulación, validación y desarrollo de algoritmos relacionados con la conducción autónoma.

Una de las principales características de RoadRunner es su capacidad para trabajar con mapas basados en el estándar ASAM OpenDRIVE y con información geográfica procedente de OpenStreetMap. Gracias a ello, es posible generar escenarios inspirados en ubicaciones reales, utilizando la red de carreteras existente como punto de partida para construir entornos virtuales que posteriormente pueden adaptarse a las necesidades de cada experimento.

Además de la creación de carreteras, la herramienta incorpora un amplio conjunto de recursos que permiten enriquecer los escenarios mediante la incorporación de edificios, vegetación, señales verticales, marcas viales, mobiliario urbano, barreras, semáforos y otros elementos característicos de un entorno urbano. Todos estos componentes pueden organizarse de forma coherente para obtener representaciones visuales que mantengan una elevada fidelidad con respecto al entorno real.

RoadRunner proporciona también herramientas para editar la geometría de las carreteras, modificar intersecciones, añadir o eliminar carriles, definir conexiones entre vías y ajustar diferentes características de la infraestructura de las vías. Estas funcionalidades permiten adaptar un escenario real a las necesidades específicas de una investigación, facilitando la creación de situaciones que serían difíciles de reproducir mediante datos geográficos sin procesar.

Otro aspecto relevante es la compatibilidad con distintos formatos de intercambio utilizados en simulación. Los escenarios desarrollados pueden exportarse empleando estándares ampliamente utilizados, como ASAM OpenDRIVE para la descripción de la red de carreteras y formatos compatibles con motores gráficos como Unreal Engine. Esta capacidad de interoperabilidad convierte a RoadRunner en una herramienta de referencia para integrar diferentes plataformas dentro de un mismo flujo de trabajo.

En combinación con [CARLA](#), RoadRunner constituye el principal entorno de edición de mapas. Mientras que [CARLA](#) proporciona el motor de simulación y Unreal Engine la edición visual y físicas, RoadRunner permite diseñar previamente los escenarios sobre los que posteriormente se desarrollarán las pruebas. De esta forma, ambas herramientas forman parte de un flujo de trabajo complementario en el que la construcción del entorno y la simulación se encuentran claramente diferenciadas en diferentes secciones.

Tabla 3.4: Principales funcionalidades de RoadRunner Utilizadas.

Funcionalidad	Categoría	Descripción
Importación de datos geográficos	Entrada de datos	Permite importar información procedente de OpenStreetMap y otros formatos compatibles para generar automáticamente la red viaria de un escenario.
Edición de carreteras	Diseño viario	Facilita la modificación de la geometría de las vías, la configuración de carriles, intersecciones y conexiones entre distintos segmentos de carretera.
Biblioteca de recursos (<i>Assets</i>)	Modelado del entorno	Incorpora una amplia colección de edificios, señales, vegetación, mobiliario urbano y otros elementos tridimensionales para enriquecer el escenario.
Colocación manual de objetos	Personalización	Permite añadir, eliminar o modificar elementos del entorno para adaptar el escenario a las necesidades específicas de cada simulación.
Edición de señalización	Infraestructura viaria	Permite incorporar señales verticales, marcas viales, semáforos y otros elementos relacionados con la regulación del tráfico.
Compatibilidad con OpenDRIVE	Interoperabilidad	Gestiona la red viaria utilizando el estándar ASAM OpenDRIVE, facilitando el intercambio de escenarios entre diferentes herramientas de simulación.
Exportación de escenarios	Integración	Permite exportar los mapas generados para su utilización en motores gráficos y simuladores compatibles, como CARLA basado en Unreal Engine.

Capítulo 4

Diseño y ejecución de escenarios urbanos predeterminados

4.1 Introducción

En este capítulo se describe el flujo de trabajo seguido para la generación, modificación y ejecución de los escenarios utilizados durante las pruebas experimentales. Se presentan las diferentes etapas necesarias para obtener entornos urbanos adaptados al objetivo general, enfocado principalmente en la modificación de mapas predeterminados disponibles en el simulador [CARLA](#).

4.2 Preparación de los escenarios predeterminados

Para comenzar la fase de desarrollo sobre los entornos predeterminados de [CARLA](#), identificados mediante la denominación *Town*, se llevó a cabo una serie de tareas de preparación orientadas a definir los escenarios de simulación que se emplearían a lo largo del desarrollo.

En primer lugar, se analizaron los doce mapas predeterminados disponibles en [CARLA](#), recogidos en la tabla [3.1](#), con el objetivo de identificar aquellos que presentaban características más adecuadas para la generación de vídeos y su posterior análisis semántico mediante modelos de lenguaje multimodales.

Una vez seleccionados los mapas más representativos, se definieron diferentes recorridos sobre cada uno de ellos. Para cada recorrido se establecieron dos variantes: una ruta de referencia, correspondiente a una situación de circulación sin incidencias, y una ruta con bloqueo, en la que la presencia de obstáculos obliga al vehículo a modificar su trayectoria original. De este modo, se generan situaciones comparables entre sí que permiten evaluar posteriormente la capacidad de los modelos para detectar y describir los cambios presentes en el entorno.

4.2.1 Selección de escenarios

Tras analizar los mapas predeterminados incluidos en [CARLA](#), se realizó una evaluación de los doce escenarios disponibles con el objetivo de determinar cuáles resultaban más adecuados para la generación de los casos de prueba. La selección no atendió únicamente al aspecto visual de los entornos, sino que consideró aquellos factores capaces de aportar mayor variedad y utilidad para el objetivo final del

proyecto: generar secuencias de conducción con bloqueos que pudieran ser analizadas posteriormente mediante modelos de lenguaje.

Los criterios principales empleados fueron la diversidad de tipos de vías, la presencia de intersecciones y zonas de circulación compleja, la cantidad de mobiliario urbano, la posibilidad de introducir obstáculos de forma realista y la capacidad del entorno para provocar modificaciones significativas en la trayectoria del vehículo.

A partir de este análisis, los escenarios seleccionados fueron Town02, Town03, Town04 y Town10, por ofrecer características complementarias entre sí y permitir crear situaciones de circulación con distintos niveles de complejidad.

- Town02 fue seleccionado por su simplicidad y estructura reducida, lo que lo convierte en un entorno idóneo para validar el correcto funcionamiento del sistema antes de abordar escenarios más exigentes. Permite comprobar la generación de obstáculos, el seguimiento de rutas y la ejecución de las simulaciones en condiciones controladas.
- Town03 destaca por la variedad de elementos urbanos que incorpora: calles de distinta anchura, intersecciones, cambios de dirección y una mayor presencia de mobiliario urbano. Esta combinación permite generar situaciones más próximas a la circulación urbana real y facilita la incorporación de bloqueos que producen modificaciones claras en la trayectoria del vehículo, lo que resulta especialmente útil para la evaluación posterior de los modelos.
- Town04 se caracteriza por sus grandes dimensiones y por incluir distintos tipos de vías, entre ellas una autopista. Esto permite estudiar situaciones en las que el vehículo circula a velocidades más elevadas y en las que los bloqueos tienen un impacto diferente al de los entornos urbanos cerrados. Además, la longitud de sus recorridos facilita la generación de trayectorias más extensas, ampliando el rango de situaciones analizables.
- Town10 fue seleccionado por su elevado nivel de detalle y complejidad. Presenta una gran cantidad de elementos urbanos, calles con configuraciones variadas y zonas de alta densidad de objetos, lo que genera escenarios más exigentes desde el punto de vista perceptivo. En este entorno, los modelos de lenguaje deben ser capaces de distinguir entre los elementos habituales del entorno y aquellos que representan un bloqueo, lo que lo convierte en el escenario más desafiante del conjunto.

Los escenarios restantes fueron descartados por presentar características menos adecuadas para los objetivos del proyecto. Town06 está compuesto principalmente por autopistas interconectadas, lo que limita la variedad de situaciones urbanas y reduce las posibilidades de generar bloqueos representativos. Town07 representa un entorno rural con escasa presencia de elementos urbanos y baja complejidad de circulación, factores que reducen su interés para el análisis semántico planteado.

Town08 y Town09, pese a presentar características interesantes, están orientados a pruebas de competición y evaluación de agentes autónomos dentro de la comunidad de [CARLA](#), por lo que su estructura no se ajusta a la generación de escenarios urbanos controlados empleada en este proyecto. Town11, por su parte, combina una red vial extensa con una baja densidad de mobiliario urbano, lo que reduce la cantidad de información visual relevante para el análisis. Su gran tamaño implica además un mayor coste computacional, dificultando la ejecución de pruebas con múltiples configuraciones de obstáculos.

La combinación final de Town02, Town03, Town04 y Town10 permite cubrir diferentes niveles de complejidad y variedad de situaciones de circulación, conformando un conjunto suficientemente

representativo para evaluar la capacidad de los modelos de lenguaje a la hora de identificar bloqueos y cambios en el entorno.



(a) Town02



(b) Town03



(c) Town04



(d) Town10

Figura 4.1: Mapas predeterminados elegidos del simulador CARLA.

4.2.2 Definición de rutas

Una vez seleccionados los escenarios sobre los que se desarrollarían las simulaciones, el siguiente paso consistió en definir un conjunto de rutas representativas que permitieran evaluar el comportamiento del vehículo en diferentes situaciones de circulación y generar los vídeos destinados al análisis mediante modelos de lenguaje.

Para cada ruta diseñada se establecieron dos variantes. La primera corresponde a una ruta de referencia u óptima, en la que el vehículo recorre el trayecto desde un punto de origen hasta un punto de destino sin encontrar ningún obstáculo que altere su recorrido. La segunda variante mantiene los mismos puntos de inicio y fin, pero incorpora uno o varios bloqueos situados estratégicamente a lo largo del trayecto, obligando al vehículo a modificar parcial o completamente la trayectoria prevista y generando así un escenario directamente comparable con la ruta original.



(a) Ruta óptima definida en Town02.



(b) Ruta alterada debido a obstáculos.

Figura 4.2: Comparación entre ruta óptima y ruta alterada por obstáculos (Town02)



(a) Ruta óptima definida en Town03.



(b) Ruta alterada debido a obstáculos

Figura 4.3: Comparación entre ruta óptima y ruta alterada por obstáculos (Town03).

Mantener el mismo origen y destino en ambas variantes garantiza que las diferencias observadas durante la simulación sean consecuencia exclusivamente de la incorporación de los bloqueos, produciendo cambios claramente identificables que facilitan el análisis semántico posterior.

El diseño de las rutas se realizó buscando representar situaciones de circulación lo más realistas y variadas posible. Con el fin de asegurar esa diversidad, los recorridos fueron definidos atendiendo a distintos niveles de complejidad, considerando aspectos como el número de intersecciones, los cambios de dirección, la posible aparición de bucles de circulación y la distancia entre puntos de toma de decisión. De este modo, los recorridos atraviesan diferentes elementos característicos de la red vial de cada escenario: intersecciones, rotondas, cambios de carril, vías de varios carriles y, en los mapas que lo permitían, tramos de autopista. Asimismo, la ubicación de los obstáculos fue cuidadosamente seleccionada para provocar modificaciones significativas en el recorrido sin llegar a impedir completamente la circulación del vehículo.

Como resultado, se obtuvo un conjunto de recorridos que cubre diferentes niveles de complejidad y una amplia variedad de situaciones de circulación, constituyendo una base adecuada para la generación de los escenarios de prueba destinados al análisis mediante modelos de lenguaje.

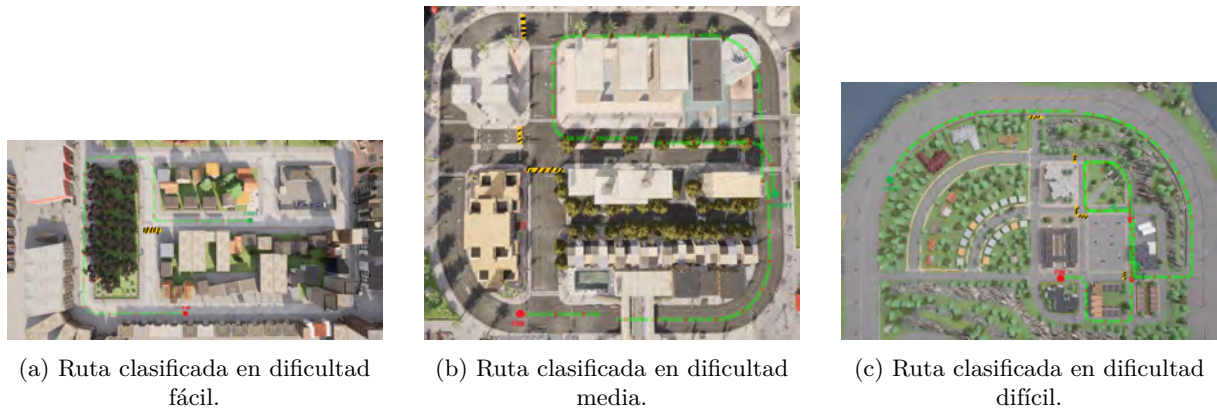


Figura 4.4: Mapas de CARLA clasificados según dificultad.

4.2.3 Selección de obstáculos

Una vez definidas las rutas de circulación, el siguiente paso consistió en seleccionar los elementos que se utilizarían para generar los distintos bloqueos presentes en los escenarios de prueba. Estos elementos, denominados *props* dentro del simulador, permiten modificar las condiciones de circulación mediante la restricción parcial o total del paso del vehículo en determinados puntos del recorrido.

La ubicación de los obstáculos se diseñó principalmente sobre intersecciones, accesos a calles y otros puntos estratégicos de la red viaria donde un corte de circulación obligase al vehículo a modificar su trayectoria. De este modo, cada bloqueo produce una variación significativa respecto a la ruta de referencia, manteniendo siempre los mismos puntos de origen y destino y permitiendo comparar ambos recorridos de forma directa.

La selección de los *props* no se realizó de manera aleatoria, sino buscando representar situaciones habituales en un entorno urbano real. Para ello, se emplearon elementos comúnmente asociados a cortes de tráfico y señalización temporal: conos de tráfico, conos luminosos, señales de advertencia por obras o accidentes, barreras de obra, balizas luminosas y otros dispositivos de señalización vial. El uso de este tipo de objetos facilita tanto la interpretación visual del escenario por parte de un observador humano como el posterior análisis semántico realizado por los modelos de lenguaje.

Con el objetivo de incrementar la diversidad de los escenarios, se incorporaron también elementos menos habituales, como contenedores, barreras de cadena o vallas metálicas, que introducen una mayor variabilidad en la apariencia de los bloqueos y añaden dificultad en su reconocimiento. Además, en determinados escenarios se recurrió a vehículos de emergencia del propio simulador, como coches patrulla, ambulancias o camiones de bomberos, con el propósito de recrear situaciones derivadas de accidentes, intervenciones policiales o emergencias, aportando un mayor grado de realismo a las simulaciones.



Figura 4.5: Catálogo de objetos más utilizados para el bloqueo de rutas.

La totalidad de los elementos empleados pertenece al catálogo de activos incluido en [CARLA](#), registrados mediante identificadores que permiten instanciarlos dinámicamente desde la API de Python durante la ejecución. Gracias a este mecanismo, los obstáculos se generan automáticamente a partir de la información almacenada en los archivos de configuración de cada escenario, sin necesidad de modificar manualmente el mapa antes de cada ejecución.

No obstante, el sistema no se limita exclusivamente a los recursos nativos del simulador. Durante el desarrollo se incorporaron activos procedentes de fuentes externas cuando resultaban más adecuados para representar determinadas situaciones o aportar mayor variedad visual. Su integración se realiza mediante la importación al proyecto de Unreal Engine y el registro de sus identificadores correspondientes, de forma que puedan instanciarse desde la API de Python del mismo modo que los activos nativos. Esta arquitectura hace que el sistema sea fácilmente extensible, permitiendo ampliar el catálogo de obstáculos disponibles sin necesidad de modificar el proceso de generación automática de escenarios.

Listado 4.1: Fragmento del archivo JSON con catalogo de *props* generables con la API.

```

1 {"props": [
2   {
3     "name": "ATM",
4     "path": "/Game/Carla/Static/Static/SM_Atm.SM_Atm",
5     "size": "Medium"
6   },
7   {
8     "name": "Barbeque",
9     "path": "/Game/Carla/Static/Dynamic/Garden/SM_Barbecue.SM_Barbecue",
10    "size": "Small"
11  },
12  {
13    "name": "Barrel",
14    "path": "/Game/Carla/Static/Dynamic/Trash/SM_Barrel.SM_Barrel",
15    "size": "Small"

```

```
16     },
17     {
18         "name": "Bench01",
19         "path": "/Game/Carla/Static/Static/SM_Bench01.SM_Bench01",
20         "size": "Medium"
21     }
22 ]
23 }
```



(a) Contenedor de gran tamaño añadido.



(b) Barrera de concreto añadida.

Figura 4.6: Ejemplo de *props* adicionales a los predeterminados de CARLA.

4.3 Configuraciones de los escenarios

Una vez definidas las rutas y seleccionados los obstáculos que formarían parte de cada escenario, fue necesario establecer un mecanismo que permitiera almacenar toda esta información de forma estructurada y reutilizable. Para ello, se optó por un sistema de configuración basado en archivos externos, evitando que los parámetros de cada escenario quedasen definidos directamente en el código de la aplicación.

Esta decisión responde a varios objetivos. En primer lugar, permite desacoplar la definición de los escenarios de la lógica de ejecución, facilitando el mantenimiento y la evolución del proyecto. Además, elimina la necesidad de modificar el código cada vez que se desea crear una nueva ruta, incorporar obstáculos adicionales o ajustar la configuración de un escenario existente. Por último, mejora la escalabilidad del sistema, ya que la incorporación de nuevos casos de prueba se reduce a la creación o modificación de archivos de configuración, sin afectar al resto de la implementación.

Cada archivo almacena toda la información necesaria para generar un escenario en tiempo de ejecución: las rutas definidas, los obstáculos que deben instanciarse y los parámetros requeridos para su posicionamiento dentro del entorno. De este modo, el sistema construye automáticamente cada escenario durante la simulación, simplificando la generación de nuevos casos de prueba y favoreciendo la automatización completa del proceso.

4.3.1 Diseño del archivo de configuración

Una vez decidido que la configuración de los escenarios se almacenaría en archivos externos, fue necesario definir una estructura que permitiera representar de forma organizada toda la información necesaria para generar cada caso de prueba durante la ejecución de la simulación.

Para ello se optó por el formato YAML, descartando otras alternativas como JSON o XML. Esta elección se justifica principalmente por la facilidad de lectura y escritura que ofrece, especialmente cuando los archivos contienen una gran cantidad de información estructurada. A diferencia de JSON o XML, YAML prescinde de caracteres de cierre y etiquetas redundantes, lo que resulta en archivos más limpios y fáciles de editar manualmente. Además, su compatibilidad con Python a través de bibliotecas estándar facilitó su integración en el sistema sin introducir dependencias adicionales.

La estructura del archivo se diseñó siguiendo una organización jerárquica en la que cada escenario constituye una unidad independiente que agrupa toda la información necesaria para su generación. De este modo, un mismo archivo puede contener múltiples escenarios, permitiendo seleccionar en cada ejecución cuál de ellos debe cargarse.

El nivel superior del archivo corresponde a la sección **scenarios**, dentro de la cual se registra cada escenario mediante un identificador único. Este identificador sigue una nomenclatura descriptiva que codifica el tipo de bloqueo, el número del escenario y el mapa de CARLA sobre el que debe generarse, lo que facilita la localización y gestión de un gran número de configuraciones. Cada escenario incorpora además una breve descripción textual que resume el tipo de bloqueo representado, útil tanto para la organización de los casos de prueba como para su identificación durante las fases de desarrollo y experimentación.

A continuación, cada escenario contiene una sección dedicada a los obstáculos que deben instanciarse. Para cada elemento se almacena el identificador del activo correspondiente dentro del catálogo de recursos de [CARLA](#), junto con la información de posicionamiento necesaria para situarlo correctamente en el entorno. Dicha información se organiza mediante una transformación **transform**, compuesta por dos campos principales:

- **location**, que especifica las coordenadas tridimensionales (X,Y,Z) del objeto dentro del mapa.
- **rotation**, que determina su orientación (pitch, yaw, roll) respecto al sistema de coordenadas del escenario, permitiendo ajustar la dirección del elemento según las características del bloqueo que se pretende representar.

Además de los obstáculos que constituyen el bloqueo propiamente dicho, el archivo contempla la generación de ciertos objetos auxiliares empleados como referencias visuales durante las pruebas como una flecha situada al inicio del recorrido para señalar el punto de partida y una señal de STOP elevada al final del trayecto para marcar el punto de destino. Estos elementos no intervienen en la circulación del vehículo, pero facilitan la validación de los escenarios.

Esta organización permite que todos los elementos necesarios para construir un escenario queden completamente definidos en el archivo de configuración, de forma que la aplicación pueda generarlos automáticamente en tiempo de ejecución sin requerir modificaciones adicionales sobre el mapa. La separación entre la lógica de generación y la definición de los escenarios simplifica el mantenimiento del sistema y agiliza la incorporación de nuevos casos de prueba.

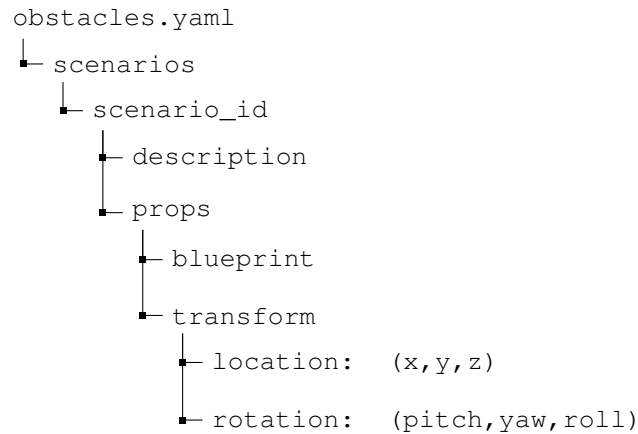


Figura 4.7: Estructura jerárquica del archivo de configuración YAML.

Listado 4.2: Fragmento del archivo YAML de configuración de props

```

1 props: #List of props
2
3   static.prop.streetbarrier:
4
5     - transform:
6       location: {x: 45.91, y: 250.181, z: 0.02}
7       rotation: {pitch: 0.0 ,yaw: 0.0 ,roll: 0.0}
8
9     - transform:
10      location: {x: 41.80 ,y: 250.20 ,z: 0.02}
11      rotation: {pitch: 0.0 ,yaw: 0.0, roll: 0.02 }
12
13   static.prop.trafficcone01:
14
15     - transform:
16       location: {x: 47.19 ,y: 250.44 ,z: 0.02 }
17       rotation: {pitch: 0.0 ,yaw: 90.0 ,roll: 0.0}
18
19     - transform:
20       location: {x: 40.60 ,y: 250.40,z: 0.03}
21       rotation: {pitch: 0.0 ,yaw: 90.0 ,roll: 0.0}

```

4.3.2 Generación dinámica de los obstáculos

Una vez definida la configuración del escenario mediante el archivo YAML, el sistema lleva a cabo una serie de procesos que permiten traducir esa información en elementos concretos dentro del entorno de simulación. El objetivo es que la generación de los obstáculos sea completamente automática a partir de los parámetros almacenados, sin requerir intervención manual entre una ejecución y la siguiente.

El proceso comienza con la inicialización del entorno de simulación. Tras el arranque de CARLA y la carga del mapa correspondiente, el sistema lee y parsea el archivo de configuración asociado al escenario seleccionado, extrayendo la información necesaria para construir cada uno de los elementos que lo componen: el identificador del activo, su posición en el mapa y su orientación dentro del entorno.

A partir de estos parámetros, el sistema instancia dinámicamente cada actor a través de la [API](#) de Python de [CARLA](#), posicionándolo en las coordenadas indicadas y aplicando la rotación definida. Este proceso se repite secuencialmente para cada obstáculo registrado en el archivo, de forma que la distribución final de los elementos sea idéntica en cada ejecución de una misma prueba, garantizando la reproducibilidad de los escenarios.

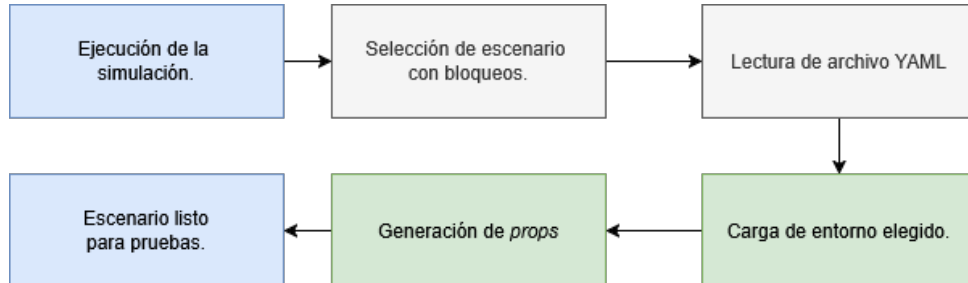


Figura 4.8: Diagrama de flujo del proceso de generación de bloqueos.

Una vez generados todos los actores, el entorno queda completamente preparado para iniciar la simulación. El vehículo puede entonces ejecutar la ruta previamente definida bajo las condiciones establecidas, permitiendo obtener recorridos con y sin presencia de obstáculos de forma sistemática y reproducible.



(a) Calle sin bloqueo (Town10)



(b) Calle bloqueada (Town10)



(c) Calle sin bloqueo (Town02)



(d) Calle bloqueada (Town02).

Figura 4.9: Comparativa entre calles sin bloqueo y calles inhabilitadas luego de generación de *props*.

la conectividad entre el origen y el destino de cada recorrido y verificar la validez de las rutas propuestas antes de incorporarlas a las simulaciones.



Figura 4.11: Establecimiento de ruta sobre *Waypoints* guía.

El resultado de este proceso es una secuencia ordenada de *Waypoints* entre el punto de inicio y el punto final de cada recorrido, que durante la simulación actúa como referencia para el sistema de conducción del vehículo. Al disponer de esta secuencia predefinida, el vehículo no necesita calcular la ruta en cada ejecución, sino que se limita a seguirla, lo que garantiza que todas las simulaciones de un mismo escenario sean reproducibles y comparables entre sí. Este enfoque permitió definir de forma sencilla tanto las rutas de referencia como las rutas modificadas por la presencia de obstáculos, asegurando que las diferencias observadas entre ambas se debieran únicamente a las condiciones del escenario y no a variaciones en la planificación del recorrido.

4.4.2 Configuraciones Traffic Manager

El *Traffic Manager* de [CARLA](#) es el componente encargado de gestionar el comportamiento de los vehículos controlados automáticamente dentro del simulador. Este módulo permite configurar diferentes aspectos relacionados con la conducción, como el seguimiento de rutas, la respuesta ante las normas de tráfico o el comportamiento durante la circulación. Su utilización resulta fundamental para obtener recorridos consistentes y reproducibles, requisito indispensable para poder comparar posteriormente las distintas simulaciones realizadas.

Con el fin de adaptar el comportamiento del vehículo a las necesidades del proyecto, fue necesario modificar diferentes parámetros del *Traffic Manager*. La Tabla 4.1 resume las principales configuraciones aplicadas y la finalidad de cada una de ellas, cuyo funcionamiento se detalla a continuación.

Tabla 4.1: Configuraciones principales aplicadas al Traffic Manager.

Configuración	Finalidad	Estado del parámetro
Modo síncrono	Garantizar que todos los elementos de la simulación (vehículos, obstáculos y sensores) evolucionen de forma coordinada, asegurando la reproducibilidad de las ejecuciones.	Activado
Ignorar determinadas señales de tráfico	Reducir la duración de los recorridos evitando detenciones innecesarias y centrar el experimento en la interacción del vehículo con los bloqueos viales.	100% / 0% (Según escenario)
Cambio automático de carril	Permitir que el vehículo pueda modificar su trayectoria cuando sea necesario para rodear un obstáculo y completar la ruta definida.	Activado
Seguimiento de rutas mediante <i>Waypoints</i>	Guiar al vehículo a lo largo de una trayectoria previamente establecida entre el origen y el destino del recorrido.	Secuencia de <i>Waypoints</i>

Una de las configuraciones más relevantes fue la activación del modo síncrono de la simulación. En este modo de funcionamiento, el servidor únicamente avanza al siguiente instante temporal cuando todos los clientes han completado las operaciones correspondientes al instante actual, lo que garantiza que la generación de los obstáculos, la actualización del estado del vehículo y la captura de imágenes se produzcan de forma sincronizada entre ejecuciones sucesivas de un mismo escenario.

En determinados escenarios se configuró también una probabilidad de ignorar ciertas señales de tráfico, como semáforos o señales de prioridad, con el objetivo de reducir el tiempo total de las simulaciones y, en consecuencia, la duración de los vídeos obtenidos. De este modo, el análisis posterior se centra en los elementos de interés del estudio: la presencia de bloqueos viales y la respuesta del vehículo ante ellos.

Asimismo, se habilitó el cambio automático de carril para los casos en que la trayectoria prevista lo requería, permitiendo que el vehículo pudiera completar aquellas rutas cuya circulación se veía alterada por la presencia de obstáculos. En conjunto, estas configuraciones permitieron obtener un comportamiento de conducción estable y repetible, garantizando que las diferencias observadas entre las simulaciones fueran consecuencia de las condiciones del escenario y no de variaciones introducidas por el sistema de control del vehículo.

4.5 Ejecución de las simulaciones

Una vez definidas las rutas de prueba, configurados los obstáculos mediante los archivos YAML y establecidos los parámetros de comportamiento del vehículo, se procede a la ejecución de las simulaciones. Esta etapa tiene como objetivo comprobar el correcto funcionamiento del escenario generado y obtener los recorridos que posteriormente serán utilizados como entrada para el análisis mediante modelos [LLM](#).

El proceso de ejecución sigue una serie de pasos definidos, representados en el diagrama de flujo mostrado en la Figura 4.13. En primer lugar, se inicia el servidor de [CARLA](#), necesario para establecer la comunicación entre el simulador y los módulos encargados de controlar la ejecución de los escenarios. Una vez activo el entorno de simulación, se selecciona la configuración correspondiente al escenario que se desea ejecutar mediante una interfaz desarrollada específicamente para este propósito, que permite

cargar de forma sencilla el archivo YAML correspondiente sin necesidad de modificarlo manualmente ni reiniciar el entorno de simulación.

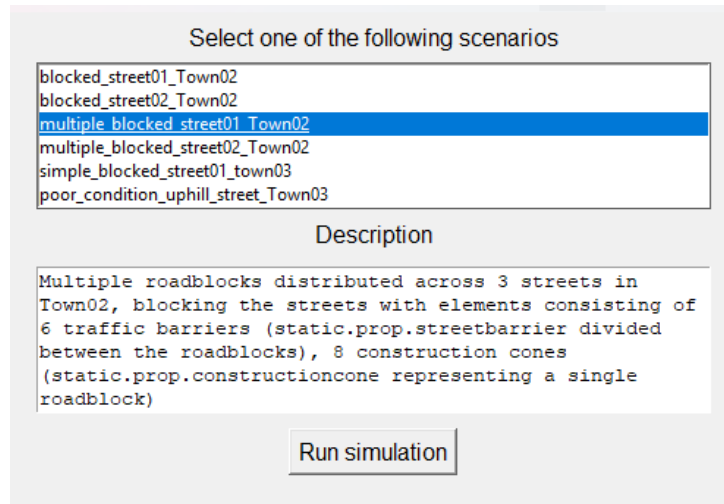


Figura 4.12: Interfaz gráfica para la selección de escenarios y bloqueos.

Tras seleccionar una configuración, el sistema carga automáticamente el mapa indicado y genera los actores asociados a los bloqueos definidos en el archivo, reconstruyendo así el escenario descrito. Esto permite visualizar previamente la distribución de los elementos dentro del mapa y verificar que el escenario se ha construido correctamente antes de iniciar la simulación.

Finalmente, con el entorno preparado y el vehículo configurado, se ejecuta el recorrido siguiendo la trayectoria establecida mediante los *waypoints*. Durante esta fase y habiendo comprobado el seguimiento de la ruta anteriormente, se realizan capturas de la simulación que posteriormente serán convertidas, obteniendo así el video correspondiente al escenario, que será utilizado posteriormente para el análisis semántico mediante LLM.

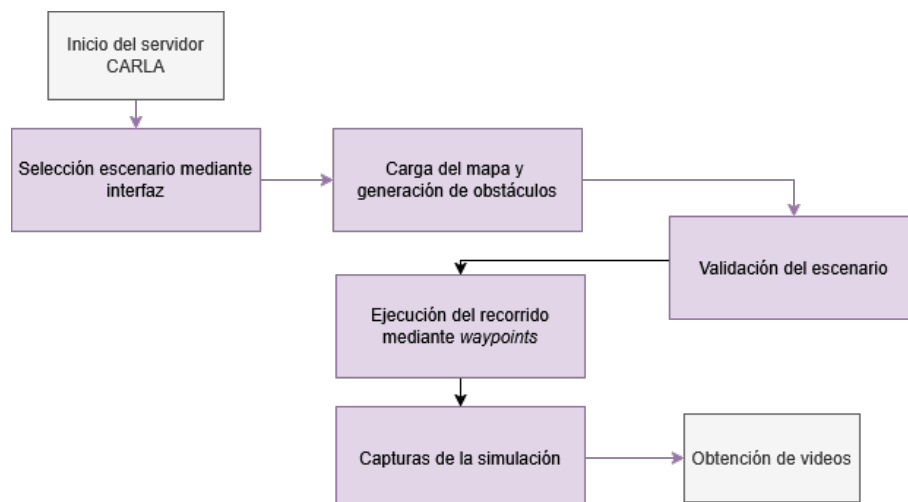


Figura 4.13: Diagrama de flujo del proceso de ejecución de una simulación hasta la obtención de videos.

4.6 Obtención de videos

Una vez configurados los escenarios y preparados los elementos necesarios para la ejecución de la simulación, se procede a la captura de información visual mediante los sensores disponibles en CARLA. En este trabajo se empleó una cámara RGB asociada al vehículo simulado, con el fin de obtener una representación visual del entorno desde la perspectiva del propio vehículo. Estas imágenes constituyen la entrada utilizada posteriormente por los modelos [LLM](#) para realizar el análisis semántico de las rutas y la identificación de los bloqueos presentes en el escenario.

La cámara se colocó en la parte superior del vehículo, a una altura determinada con el objetivo de evitar que elementos pertenecientes al propio coche apareciesen dentro del campo de visión. De este modo, las imágenes obtenidas se centran exclusivamente en la escena urbana situada delante del vehículo, reduciendo información visual no relevante y facilitando el análisis posterior.

Para la adquisición de las imágenes se estableció una resolución de 1280×720 píxeles, suficiente para conservar el nivel de detalle necesario para identificar los elementos presentes en la escena, sin generar un volumen de datos excesivo para su almacenamiento y procesamiento. Asimismo, se configuró un campo de visión (Field Of View (FOV)) de aproximadamente 110 grados, lo que permite capturar tanto la zona frontal de circulación como parte del entorno lateral, necesaria para analizar las modificaciones en la trayectoria provocadas por la presencia de obstáculos.



Figura 4.14: [Field Of View \(FOV\)](#) de vehículo utilizado para la captura de imágenes.

Respecto a la frecuencia de captura, la cámara se configuró para obtener una imagen cada 0.35 segundos, lo que permite generar una secuencia temporal representativa del recorrido del vehículo manteniendo un equilibrio entre la cantidad de información capturada y el volumen de datos generado durante las simulaciones.

Una vez finalizada la ejecución de cada escenario, las imágenes obtenidas se procesaron mediante la herramienta *FFmpeg*, encargada de convertir la secuencia de imágenes individuales en un archivo de vídeo con una tasa de salida de entre 8 y 10 fotogramas por segundo, suficiente para obtener una representación fluida del movimiento del vehículo a partir de imágenes capturadas a menor frecuencia.

El formato de salida seleccionado fue *.mp4*, por su amplia compatibilidad y facilidad de manipulación en distintas herramientas de análisis. Los vídeos obtenidos constituyen el resultado final de la fase de simulación y la entrada utilizada posteriormente por los modelos de lenguaje para evaluar la comprensión

del entorno, identificar la presencia de bloqueos y analizar las modificaciones producidas en la ruta del vehículo.

Tabla 4.2: Parámetros de configuración de la cámara y del proceso de generación de vídeo.

Parámetro	Finalidad	Valor
Resolución	Conservar el nivel de detalle necesario sin generar un volumen de datos excesivo.	1280 × 720 px
Campo de visión (FOV)	Capturar la zona frontal y parte del entorno lateral del vehículo.	~110°
Frecuencia de captura	Generar una secuencia temporal representativa del recorrido.	1 imagen / 0.35 s
Tasa de fotogramas de salida	Obtener una representación fluida del movimiento del vehículo.	8 – 10 fps
Formato de salida	Garantizar compatibilidad y facilidad de manipulación posterior.	.mp4

Capítulo 5

Generación de escenarios urbanos personalizados

5.1 Introducción

En este capítulo se desarrolla el segundo enfoque de trabajo centrado en la generación de entornos urbanos personalizados a partir de datos geográficos reales. El proceso parte de la información cartográfica obtenida desde [Open Street Map \(OSM\)](#), que es posteriormente procesada mediante RoadRunner para construir la red viaria y los principales elementos del escenario urbano.

Una vez generado el mapa base, este se integra en Unreal Engine, donde se incorporan elementos adicionales del entorno con el fin de aumentar el nivel de detalle y el realismo visual de la escena. El escenario resultante queda finalmente preparado para su uso en [CARLA](#), donde se aplica el mismo procedimiento de definición de rutas, incorporación de obstáculos y ejecución de simulaciones descrito en el capítulo anterior.

5.2 Obtención de datos cartográficos

La generación de escenarios urbanos personalizados comienza con la obtención de los datos cartográficos correspondientes a la zona que se desea recrear. Estos datos constituyen la base sobre la que posteriormente se construirá el entorno de simulación, ya que contienen la información necesaria para definir la red viaria y otros elementos geográficos del escenario.

Para ello, se empleó [Open Street Map \(OSM\)](#) como fuente de información geográfica. Tal y como se describió en capítulos anteriores, esta plataforma proporciona datos cartográficos de libre acceso que permiten obtener la información necesaria para reconstruir entornos urbanos reales y utilizarlos como punto de partida en el proceso de generación del mapa.

5.2.1 Selección del área urbana

La selección de las áreas de estudio se realizó atendiendo a criterios de diversidad de la red viaria y al potencial de cada zona para la generación de distintos escenarios de simulación. En lugar de seleccionar zonas urbanas homogéneas, se priorizaron entornos que incorporasen diferentes tipologías

de vías, intersecciones y edificaciones, permitiendo posteriormente introducir modificaciones sobre la infraestructura de carreteras y generar un mayor número de casos de prueba.

Como primer escenario se seleccionó la zona suroeste del pueblo de *Meco*, con una superficie aproximada de 0.52 km². Esta área presenta una red de carreteras de complejidad fácil-media, compuesta por diferentes tipos de intersecciones, glorietas, calles sin salida y vías residenciales. La distribución de las edificaciones y la variedad de configuraciones de las carreteras permiten introducir modificaciones de forma sencilla, como la sustitución de glorietas por intersecciones convencionales, la creación de nuevas conexiones entre calles o la reorganización de determinados accesos, lo que convierte a este escenario en un entorno adecuado para las primeras pruebas de generación y validación de mapas personalizados.

El segundo escenario corresponde a una zona urbana de mayor extensión y complejidad, situada en *Zona de espartales y parte del campus universitario*, con una superficie total de aproximadamente 2.27 km². A diferencia del escenario anterior, este entorno combina áreas de elevada densidad de edificaciones con espacios abiertos y zonas de menor presencia de elementos urbanos, e incorpora además una mayor variedad de señalización, accesos a vías principales e intersecciones de distinta naturaleza. Estas características lo convierten en un entorno más exigente para la generación de escenarios personalizados, además de permitir disponer de un número considerablemente mayor de configuraciones y recorridos posibles gracias a su mayor extensión.

En ambos escenarios se exportaron los datos cartográficos necesarios para reconstruir el entorno urbano, incluyendo la geometría de la red viaria, las intersecciones, las parcelas y el resto de elementos geográficos empleados posteriormente en el proceso de generación del mapa en RoadRunner. La Figura 5.1 muestra las áreas seleccionadas para cada uno de los escenarios utilizados durante el desarrollo del proyecto.



(a) Datos geográficos del primer escenario.

(b) Datos geográficos del segundo escenario.

Figura 5.1: Obtención de los datos geográficos para la generación de escenarios.

5.2.2 Exportación de datos geográficos

Una vez seleccionadas las áreas de estudio, el siguiente paso consiste en exportar la información cartográfica correspondiente para su utilización en las etapas posteriores del proceso de generación del escenario. OpenStreetMap permite obtener los datos geográficos de la zona seleccionada en formato *.osm*, que constituye el formato de intercambio empleado en este trabajo.

El archivo *.osm* almacena la información necesaria para reconstruir la estructura básica del entorno urbano, incluyendo la geometría de la red de carreteras y la disposición de los principales elementos cartográficos presentes en la zona seleccionada. Esta información sirve como punto de partida para la

generación del escenario, ya que será posteriormente interpretada por RoadRunner para construir el modelo tridimensional del entorno.

Dado que RoadRunner admite la importación directa de archivos en formato *.osm*, no fue necesario aplicar ningún procesamiento adicional sobre los datos exportados antes de incorporarlos al flujo de trabajo. Esto permite que la información obtenida desde OpenStreetMap se mantenga íntegra, conservando la estructura original de la red de carreteras y simplificando la creación de escenarios urbanos basados en ubicaciones reales.

5.3 Construcción base del escenario mediante RoadRunner

Una vez obtenida la información cartográfica en formato *.osm*, el siguiente paso consiste en su importación dentro de la herramienta RoadRunner, con el objetivo de transformar los datos geográficos obtenidos desde OpenStreetMap en un escenario urbano preparado para su utilización en el entorno de simulación.

Este proceso comprende varias fases sucesivas: la importación del mapa base, la generación de la red vial a partir de dicha información, la incorporación de elementos urbanos básicos que complementen la infraestructura y, finalmente, la exportación del escenario resultante hacia Unreal Engine. Cada una de estas fases se describe con mayor detalle en los siguientes apartados.

5.3.1 Importación del mapa base

RoadRunner permite importar de forma directa archivos en formato *.osm* mediante la herramienta *SD Map Viewer Tool*, integrada en el propio entorno de desarrollo. Esta herramienta interpreta la información cartográfica contenida en el archivo exportado desde OpenStreetMap y genera una representación preliminar de la red de carreteras, que servirá como base para la construcción del escenario tridimensional.

Durante la importación, RoadRunner muestra una visualización inicial de la geometría de las carreteras que representa un vector de carreteras en 2D formado por diferentes nodos, permitiendo comprobar que la información cartográfica ha sido interpretada correctamente antes de comenzar el proceso de edición. La Figura 5.2 muestra un ejemplo de esta representación inicial.



Figura 5.2: Previsualización de la red de carreteras contenidas en el archivo *.osm* importado en RoadRunner.

No obstante, la red de carreteras generada a partir de los datos de OpenStreetMap requiere habitualmente una fase de revisión y depuración. Dado que la exportación se realiza sobre un área rectangular, algunas carreteras quedan parcialmente incluidas dentro de los límites de la zona seleccionada, lo que puede dar lugar a segmentos de vía incompletos, conexiones interrumpidas o tramos aislados que no forman parte de la red principal.

Para garantizar la consistencia del escenario, se realizó un proceso de limpieza sobre la red importada, que consistió en eliminar los segmentos no relevantes para el entorno final, corregir las conexiones entre carreteras cuando era necesario y suprimir las vías que permanecían aisladas o sin continuidad dentro del mapa. El resultado es una red de carreteras coherente y completamente conectada, adecuada para continuar con las siguientes fases del proceso de generación del escenario.

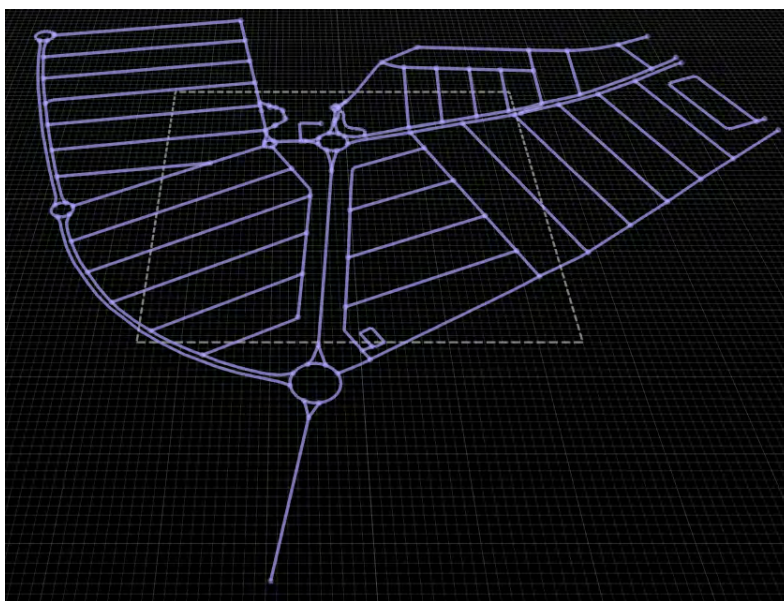


Figura 5.3: Previsualización de la red de carreteras luego de limpieza.

5.3.2 Configuración de la red vial

Depurada la red de carreteras importada desde OpenStreetMap, el siguiente paso consiste en generar la geometría tridimensional del escenario. Para ello se empleó la herramienta *SD Builder*, integrada en RoadRunner, encargada de construir automáticamente la representación física y visual del entorno a partir de la información cartográfica disponible. Como resultado, se generan los principales elementos que conforman el escenario urbano: la red de carreteras, glorietas, intersecciones, parcelas y la señalización horizontal básica.

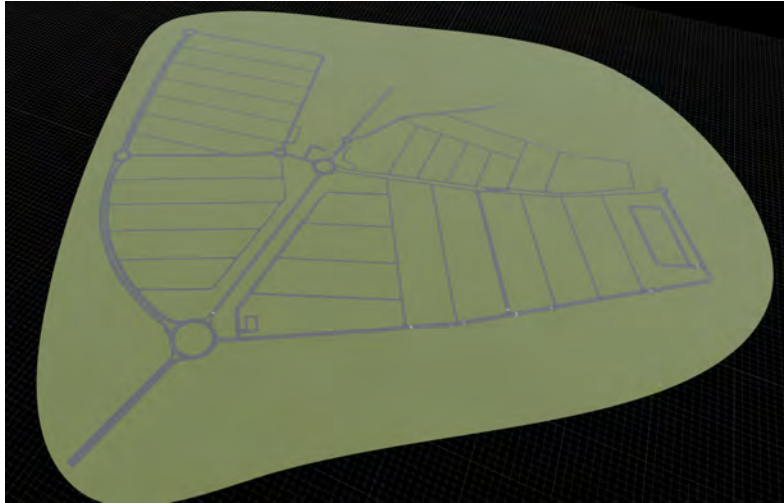
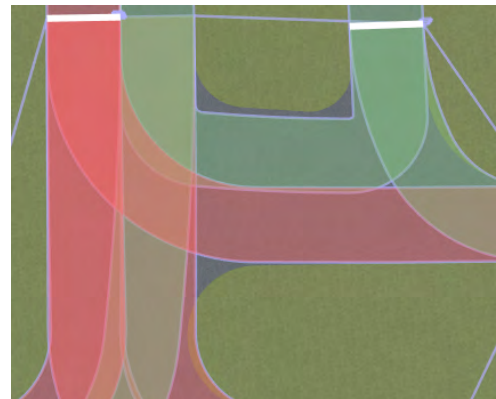


Figura 5.4: Entorno generado en base al archivo .osm con la herramienta SD Builder.

No obstante, la generación automática proporciona únicamente una base funcional sobre la que continuar el desarrollo. Durante el proceso pueden aparecer inconsistencias en la geometría de determinadas carreteras, conexiones incorrectas entre intersecciones, dimensiones poco realistas en algunos carriles o trayectorias de giro que no representan adecuadamente la distribución de la red viaria original. Por este motivo, fue necesario llevar a cabo diferentes correcciones manuales con el fin de obtener un entorno visualmente coherente y preparado para su utilización en el simulador.



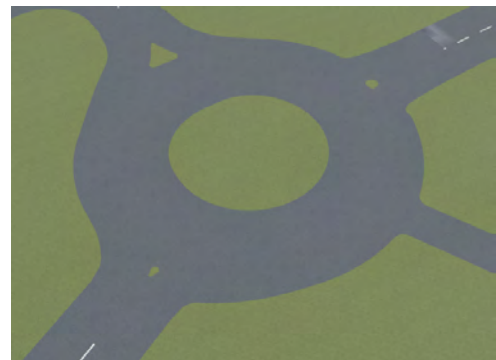
(a) Intersecciones mal conectadas



(b) Intersecciones arregladas



(c) Rotonda deformada



(d) Rotonda corregida

Figura 5.5: Antes y después de aplicar correcciones a inconsistencias generadas.

Además de estas correcciones, se incorporaron elementos urbanos básicos que complementan la infraestructura vial generada automáticamente, como aceras, bordillos, pasos de peatones y marcas viales adaptadas a la señalización habitual en entornos europeos, mejorando así la fidelidad visual del escenario sin alterar la estructura principal de la red de carreteras.

Finalizada la edición de la geometría, se procedió a revisar la información interna asociada al mapa, especialmente la relacionada con la definición de la red OpenDRIVE. Esta fase resulta fundamental, ya que los agentes de conducción de [CARLA](#) utilizan dicha información para planificar sus trayectorias. Una carretera puede estar correctamente representada desde el punto de vista visual y, sin embargo, presentar discontinuidades en la red OpenDRIVE que impidan al vehículo circular por ella. Para evitar este tipo de problemas, se revisaron y corrigieron las conexiones entre carreteras, las direcciones de circulación, los límites de velocidad, las maniobras permitidas en cada intersección y los sentidos de circulación de cada una de las carreteras involucradas en el entorno.

Una vez validada la estructura del mapa, se realizaron también modificaciones deliberadas sobre la red de carreteras con el objetivo de generar variantes del escenario original. Entre estas modificaciones se incluyen la eliminación de determinadas conexiones entre calles, la creación de nuevos enlaces y la sustitución de gloriets por intersecciones convencionales, así como el proceso inverso. Estas alteraciones permiten construir múltiples escenarios de prueba a partir de una misma base cartográfica, sin necesidad de generar un nuevo mapa desde cero.

Durante el desarrollo se comprobó que la incorporación de determinados elementos urbanos directamente desde RoadRunner presentaba problemas de compatibilidad con [CARLA](#). En particular, la generación automática de señales de tráfico y semáforos provocaba inconsistencias durante la ejecución de las simulaciones, con elementos duplicados u orientaciones incorrectas que afectaban al comportamiento del tráfico. Debido a estas limitaciones, se optó por realizar la incorporación de este tipo de objetos en Unreal Engine, donde es posible controlar con mayor precisión tanto su ubicación como su configuración dentro del escenario.

5.3.3 Exportación del escenario

Finalizada la construcción y validación del escenario en RoadRunner, el siguiente paso consiste en preparar los archivos necesarios para su integración en Unreal Engine. Esta fase permite trasladar tanto la representación visual del entorno como la información asociada a la red de carreteras, de manera que el escenario pueda ser utilizado posteriormente dentro de [CARLA](#).

RoadRunner incorpora herramientas de exportación compatibles con Unreal Engine que automatizan este proceso, generando los archivos necesarios para reconstruir el escenario en el motor gráfico. Entre ellos destacan dos archivos principales: el modelo tridimensional del entorno, almacenado en formato *.fbx*, y la descripción de la red viaria en formato *.xodr*.

El archivo *.fbx* contiene la geometría tridimensional del escenario, incluyendo las carreteras, intersecciones, parcelas y el resto de elementos visuales generados durante la fase de modelado. Este archivo es el encargado de trasladar la representación física del entorno a Unreal Engine, donde posteriormente podrá complementarse con nuevos elementos urbanos y materiales adicionales.

Por otra parte, el archivo *.xodr* almacena la descripción de la red viaria siguiendo el estándar OpenDRIVE. En él se define la estructura lógica del escenario, incluyendo la disposición de los carriles, las conexiones entre carreteras, las direcciones de circulación y el resto de información utilizada por los agentes de conducción para planificar sus trayectorias dentro del simulador. Como consecuencia,

ambos archivos son complementarios: mientras que el archivo *.fbx* proporciona la representación visual del escenario, el archivo *.xodr* define el comportamiento funcional de la infraestructura vial.

La exportación conjunta de ambos archivos constituye el punto de enlace entre RoadRunner y Unreal Engine, permitiendo continuar con el proceso de enriquecimiento del escenario antes de su integración definitiva en [CARLA](#).

5.4 Adaptación del escenario en Unreal Engine

Una vez exportado el escenario desde RoadRunner, el siguiente paso consiste en su integración dentro de Unreal Engine. Aunque la geometría del entorno y la red viaria ya se encuentran definidas, el escenario obtenido representa únicamente una base sobre la que es necesario incorporar nuevos elementos antes de su utilización en [CARLA](#).

En esta fase se lleva a cabo el enriquecimiento del entorno mediante la incorporación de elementos urbanos tanto visuales como funcionales. Gracias a las herramientas proporcionadas por Unreal Engine, es posible añadir una mayor variedad de objetos urbanos, ajustar su distribución dentro del escenario y configurar aquellos elementos relacionados con la regulación del tráfico, como la señalización vertical o los sistemas semafóricos. Estas tareas permiten obtener escenarios más realistas y, al mismo tiempo, garantizar un comportamiento adecuado de la circulación durante la ejecución de las simulaciones evitando posibles problemas de compatibilidad.

5.4.1 Importación en Unreal Editor

Una vez exportado el escenario desde RoadRunner, el siguiente paso consiste en importarlo dentro del proyecto de CARLA en Unreal Engine. Para ello, los archivos generados durante la exportación se copian en el directorio *Import* del proyecto, organizándolos en una carpeta específica para cada escenario. Esta organización facilita la gestión de los diferentes mapas desarrollados durante el proyecto y evita posibles conflictos entre archivos pertenecientes a distintas versiones del mismo.

Además de los archivos exportados por RoadRunner, es necesario incorporar un archivo de configuración en formato *.json* cuyo nombre debe coincidir con el del escenario que se desea importar. Este archivo contiene la información requerida por las herramientas de importación de CARLA para localizar y procesar correctamente los recursos asociados al mapa.

Con la estructura de directorios preparada, el proceso de importación se lleva a cabo mediante las herramientas proporcionadas por CARLA, que procesan automáticamente los archivos exportados y generan los recursos necesarios para su utilización dentro de Unreal Engine. Durante el desarrollo se detectó una incidencia relacionada con la ejecución del comando de importación, ya que este no se encontraba correctamente registrado en el entorno de trabajo. Para resolverlo se optó por ejecutar manualmente el script de importación desde la línea de comandos, lo que permitió completar el proceso satisfactoriamente.

Finalizada la importación, se verificó que el escenario se abría correctamente desde Unreal Engine y que todos los elementos generados en las etapas anteriores estaban presentes en el mapa, confirmando que tanto la geometría del escenario como la información de la red viaria habían sido procesadas correctamente. Con esto, el entorno quedó preparado para las tareas de edición y enriquecimiento descritas en los apartados siguientes.

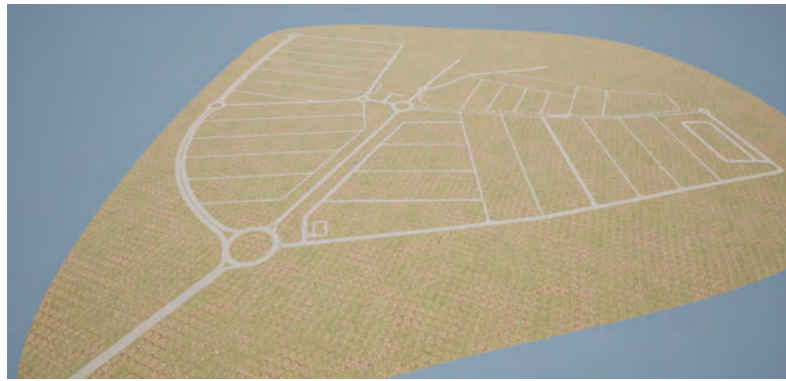


Figura 5.6: Entorno cargado dentro de Unreal Editor.

5.4.2 Ajustes del entorno

Confirmada la correcta importación del escenario, se procedió a la fase de enriquecimiento del entorno dentro de Unreal Engine. El objetivo principal de esta etapa fue incrementar el nivel de detalle del escenario y aproximar su apariencia a la de la ubicación real de la que procedían los datos cartográficos, obteniendo un entorno visualmente más completo y adecuado para la realización de simulaciones.

Las primeras modificaciones se orientaron a mejorar el aspecto de la infraestructura vial. Para ello, los materiales generados automáticamente por RoadRunner fueron sustituidos por otros disponibles en Unreal Engine, aplicando nuevas texturas sobre las carreteras, aceras y demás superficies del entorno. Asimismo, se incorporó un sistema básico de iluminación que permite contemplar la realización de simulaciones bajo diferentes condiciones de luminosidad, como escenarios nocturnos o con visibilidad reducida. Aunque estas condiciones no forman parte de los experimentos desarrollados en este trabajo, su incorporación facilita la ampliación del sistema en futuras investigaciones.

Posteriormente se procedió a poblar el escenario con una amplia variedad de elementos urbanos, empleando los recursos gráficos proporcionados por [CARLA](#) y Unreal Engine. Entre los elementos incorporados se incluyen edificaciones, vegetación, mobiliario urbano, zonas de estacionamiento, hospitales, centros universitarios, estaciones de servicio y otros componentes representativos de un entorno urbano real. En los casos en que fue necesario, también se ajustó la escala o la disposición de determinados objetos con el objetivo de mantener la coherencia visual del escenario y lograr una representación más fiel de la zona original.

Además de los elementos puramente decorativos, fue necesario incorporar aquellos componentes que intervienen directamente en el comportamiento de la circulación: señales de prioridad, señales de limitación de velocidad, señales de dirección obligatoria, prohibiciones de circulación y sistemas de semáforos. A diferencia de los elementos visuales, estos objetos incorporan información funcional que puede ser interpretada por los agentes de conducción de [CARLA](#), condicionando el comportamiento del vehículo durante la simulación.

Para garantizar su correcto funcionamiento, fue necesario configurar el área de influencia asociada a cada uno de estos elementos. Esta área determina el tramo de vía sobre el que actúa cada señal o semáforo y es utilizada por los agentes de conducción para interpretar las restricciones de circulación presentes en el escenario. Un ajuste adecuado de estos parámetros resulta esencial para que el vehículo responda correctamente ante señales de tráfico, límites de velocidad e intersecciones reguladas.

Una vez completada la incorporación y configuración de todos los elementos urbanos, se realizaron pruebas de validación para comprobar tanto la correcta visualización del escenario como

el comportamiento funcional de los componentes añadidos, confirmando que el entorno se encontraba preparado para su utilización en [CARLA](#).



Figura 5.7: Conjunto de tomas de diferentes ángulos de los escenarios poblados.

5.5 Validación de los escenarios generados

Una vez finalizadas las fases de importación y enriquecimiento del escenario en Unreal Engine, se llevó a cabo un proceso de validación para comprobar que el mapa generado podía utilizarse correctamente dentro de [CARLA](#). Esta fase tuvo como objetivo verificar tanto la correcta integración del escenario en el simulador como el funcionamiento de los distintos elementos incorporados durante las etapas anteriores.

En primer lugar, se comprobó que el escenario pudiera cargarse correctamente tras el inicio del simulador y que fuese accesible mediante la API de Python de [CARLA](#). Esta verificación confirmó que el mapa había sido integrado satisfactoriamente y que podía seleccionarse de forma dinámica durante la ejecución de las simulaciones, del mismo modo que cualquiera de los escenarios predeterminados incluidos en el simulador.

A continuación, se verificó la correcta generación de la red de *waypoints* asociada al archivo OpenDRIVE. Dado que estos puntos de referencia constituyen la base utilizada por los agentes de conducción para la planificación y seguimiento de rutas, su disponibilidad resulta indispensable para el funcionamiento del sistema dentro del escenario personalizado.

Posteriormente se realizaron recorridos de prueba siguiendo el mismo procedimiento empleado en los mapas predeterminados de [CARLA](#). Para ello, se definieron diversas rutas distribuidas por distintas zonas del escenario y se ejecutaron simulaciones con el vehículo autónomo, comprobando que el agente de conducción interpretaba correctamente la información contenida en el archivo *.xodr* y era capaz de recorrer la red viaria sin incidencias siguiendo las trayectorias definidas.

Durante este proceso se validó también el funcionamiento de los elementos funcionales incorporados en Unreal Engine. En particular, se verificó que el agente reconociese correctamente las áreas de influencia asociadas a señales de tráfico y sistemas semafóricos, respetando las prioridades de paso, las señales de *Stop*, los *Ceda el paso*, las limitaciones de velocidad y el resto de restricciones presentes en el escenario. La respuesta correcta del vehículo ante estos elementos confirmó que tanto la información visual como la funcional del entorno habían sido integradas satisfactoriamente.

La superación de todas estas verificaciones permitió considerar el escenario completamente operativo, pudiendo aplicar sobre él el mismo procedimiento de definición de rutas, incorporación de obstáculos y generación de vídeos descrito para los mapas predeterminados de [CARLA](#).

Tabla 5.1: Comprobaciones realizadas durante la validación del escenario generado.

Verificación	Objetivo	Resultado
Carga del mapa en CARLA	Comprobar la correcta integración del escenario	Correcto
Acceso mediante la API de Python	Permitir la selección dinámica del mapa	Correcto
Generación de <i>Waypoints</i>	Validar la red OpenDRIVE	Correcto
Seguimiento de rutas	Verificar la navegación del agente	Correcto
Reconocimiento de señales	Comprobar el comportamiento ante la señalización	Correcto
Funcionamiento de semáforos	Validar la regulación del tráfico	Correcto



(a) *Waypoints* en el escenario personalizado A



(b) *Waypoints* en el escenario personalizado B

Figura 5.8: Validación de funcionamiento de escenarios con la aparición de *Waypoints* en los entornos personalizados.

Capítulo 6

Análisis de escenarios mediante modelos LLM

6.1 Introducción

En este punto del desarrollo del proyecto, los escenarios predeterminados de [CARLA](#) han sido modificados mediante la incorporación de distintos elementos bloqueantes capaces de alterar las trayectorias inicialmente definidas para los vehículos simulados. Del mismo modo, se han generado nuevos escenarios a partir de la información cartográfica obtenida de OpenStreetMap, incorporando el mobiliario urbano correspondiente y configurando las normas de circulación necesarias para su correcta simulación. Tras verificar el correcto funcionamiento de todos los escenarios y de las rutas definidas, se procede a la fase de análisis mediante modelos de lenguaje multimodales.

En este capítulo se describe el proceso seguido para evaluar los distintos modelos considerados, con el objetivo de seleccionar aquel que ofrece un mejor rendimiento en el análisis semántico de los vídeos generados durante las simulaciones. Asimismo, se presenta el diseño del *prompt* empleado para guiar el comportamiento de los modelos, así como la metodología y las métricas utilizadas para comparar su desempeño y mostrar los resultados finales obtenidos.

6.2 Preparación de los datos

Antes de realizar el análisis mediante los modelos de lenguaje, fue necesario preparar adecuadamente la información que estos recibirían como entrada. Para ello, se emplearon los vídeos obtenidos durante las simulaciones, capturados mediante las cámaras incorporadas al simulador [CARLA](#). A partir de este material se definió el formato de entrada utilizado y se estableció el procedimiento de preparación de los datos, con el objetivo de proporcionar a todos los modelos una información homogénea que permitiera realizar una comparación objetiva de sus capacidades.

6.2.1 Formato de entrada de los modelos

Aunque los escenarios de simulación fueron almacenados inicialmente como archivos de vídeo en formato *.mp4* luego del procesamiento con FFmpeg, el análisis realizado por los modelos de lenguaje se llevó a cabo mediante el procesamiento de una secuencia de fotogramas extraídos periódicamente de dichos vídeos.

Con el objetivo de reducir el coste computacional sin perder información relevante sobre la evolución de la ruta, se seleccionó un fotograma de cada cinco capturados durante la simulación, garantizando así unas condiciones homogéneas para la comparación de resultados entre modelos.

Durante las primeras pruebas se observó que algunos modelos presentaban dificultades para identificar correctamente objetos situados en los extremos de la escena o para determinar su posición relativa respecto al vehículo, especialmente al distinguir entre las zonas izquierda, central y derecha de la imagen. Con el fin de mejorar la interpretación espacial, cada fotograma fue complementado con tres vistas adicionales obtenidas al dividir la imagen en regiones verticales correspondientes a dichas zonas. De este modo, para cada instante analizado se proporcionó al modelo la imagen completa junto con una versión ampliada de cada una de las tres regiones, permitiendo combinar una visión global del entorno con un mayor nivel de detalle en cada sección de la escena.

Dado que el objetivo era evaluar la capacidad de los modelos para identificar obstáculos e interpretar los cambios producidos en la circulación, únicamente se emplearon secuencias correspondientes a rutas en las que la presencia de bloqueos obligaba al vehículo a modificar su trayectoria. Todos los modelos fueron evaluados sobre este mismo conjunto de fotogramas y con los mismos *prompts*, asegurando que los resultados obtenidos fueran directamente comparables entre sí.

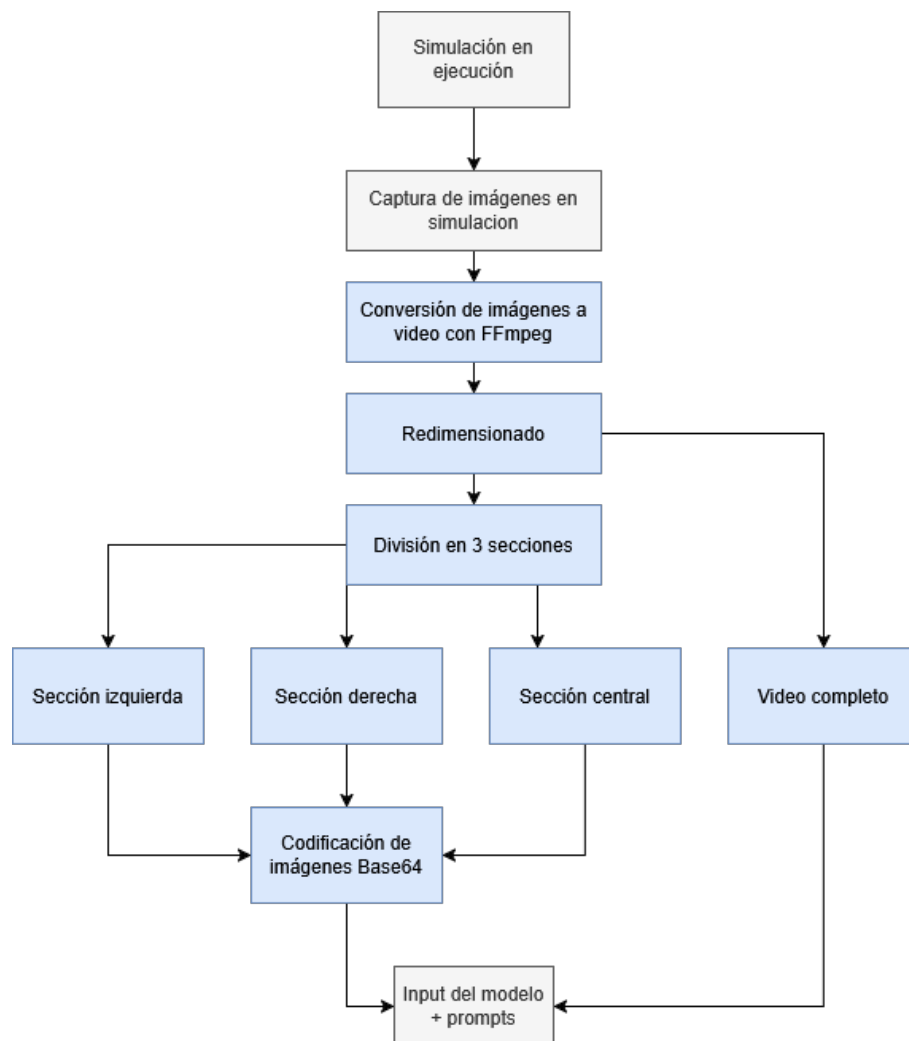


Figura 6.1: Flujo de procesamiento del vídeo para la inferencia con LLM.



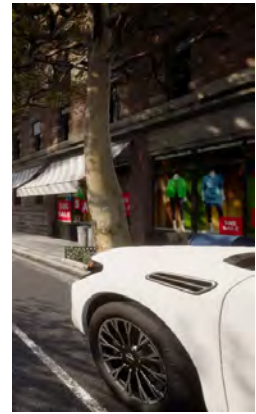
(a) Imagen completa



(b) Sección izquierda



(c) Sección central



(d) Sección derecha

Figura 6.2: Ejemplo de imagen completa y sus tres secciones codificadas y ampliadas.

6.2.2 Diseño del prompt

El rendimiento de los modelos multimodales depende en gran medida de la forma en que se les presenta la tarea. Por este motivo, se diseñó un *prompt* específico cuyo objetivo era guiar al modelo durante el análisis de cada escena y reducir, en la medida de lo posible, las ambigüedades en las respuestas generadas. Su desarrollo se realizó de forma iterativa, refinando progresivamente las instrucciones a partir de los resultados obtenidos en las sucesivas pruebas experimentales.

Para la construcción del *prompt* se emplearon diversas técnicas de *prompt engineering*. En primer lugar, se aplicó **Task Decomposition**, dividiendo el problema en subtarefas claramente definidas. En lugar de solicitar una descripción general de la escena, se indicó explícitamente al modelo que analizara de forma independiente la región izquierda, la región central y la región derecha de la vía, determinando para cada una de ellas la existencia de una carretera transitable y la presencia o ausencia de un bloqueo.

Asimismo, se incorporaron estrategias de **Negative Prompting**, especificando explícitamente los elementos que no debían considerarse obstáculos. Durante las primeras pruebas se observó que algunos modelos clasificaban incorrectamente señales de tráfico, aceras, edificios, vegetación u otros elementos situados fuera de la calzada como bloqueos que impedían la circulación. Para minimizar estos errores, se añadieron instrucciones negativas indicando qué objetos debían ignorarse y cuáles podían considerarse bloqueos reales únicamente cuando se encontraban sobre la trayectoria del vehículo.

Por último, se empleó **Structured Output Prompting**, obligando al modelo a devolver la información en un formato JSON predefinido. Cada respuesta debía incluir, para cada una de las tres regiones analizadas, el estado de la vía (bloqueada, no bloqueada o desconocida), una estimación de confianza entre 0,1 y 1,0 y una breve justificación de la decisión tomada. Este formato estructurado

permitió automatizar el procesamiento posterior de las respuestas y facilitar la comparación objetiva entre los distintos modelos evaluados.

Las sucesivas iteraciones del *prompt* permitieron reducir algunos de los errores más frecuentes observados durante las primeras pruebas: disminuyó la tendencia de los modelos a identificar como bloqueos objetos situados fuera de la calzada, se redujo el uso excesivo de la categoría *unknown* en situaciones con información suficiente para emitir un juicio y se obtuvo un comportamiento más consistente entre diferentes ejecuciones. El resultado fue un *prompt* capaz de guiar el análisis de forma más precisa, proporcionando respuestas homogéneas y adecuadas para la evaluación comparativa de los modelos.

Tabla 6.1: Componentes principales del *prompt* empleado durante la evaluación.

Sección	Descripción
Contexto	Se define el rol del modelo como analizador de escenarios de conducción autónoma encargado de determinar la existencia de rutas transitables y detectar posibles bloqueos.
Entradas	Se proporciona una imagen completa de la escena junto con tres regiones ampliadas correspondientes a la zona izquierda, central y derecha del entorno.
Instrucciones	Se solicita analizar cada región de forma independiente, determinando si existe una vía transitable y si ésta se encuentra bloqueada.
Restricciones	Se especifican los elementos que no deben considerarse bloqueos (señales, edificios, aceras, vegetación, etc.) y aquellos que sí constituyen un obstáculo real cuando impiden el paso del vehículo.
Criterios de decisión	Se prioriza la utilización de respuestas afirmativas o negativas frente a estados de incertidumbre cuando la información visual es suficiente.
Formato de salida	La respuesta debe generarse exclusivamente en formato JSON, incluyendo para cada región el estado de bloqueo, un valor de confianza y una breve justificación.

6.3 Modelos multimodales evaluados

Con el objetivo de evaluar diferentes alternativas bajo las mismas condiciones experimentales, todos los modelos multimodales empleados en este trabajo fueron ejecutados localmente. Para ello se utilizó la herramienta *LM Studio*, que proporciona un entorno unificado para la descarga, gestión y ejecución de modelos de lenguaje de código abierto, además de ofrecer una API compatible con el estándar de OpenAI que facilita su integración con aplicaciones desarrolladas en Python.

La utilización de esta herramienta permitió simplificar el proceso de instalación y configuración de los modelos, evitando la necesidad de emplear diferentes repositorios o procedimientos específicos para cada uno de ellos. Asimismo, al ejecutarse todos los modelos en el mismo entorno de trabajo, se garantizó una metodología homogénea durante la fase experimental y se facilitó la automatización del proceso de evaluación.

Los modelos seleccionados para el análisis semántico de los escenarios fueron *Qwen3-VL*, *InternVL3*, *Qwen2.5-VL* y *LLaVA 1.6*. La elección de estos modelos se realizó atendiendo a su disponibilidad para ejecución local, sus capacidades multimodales y su idoneidad para tareas de comprensión de imágenes, aspectos fundamentales para el análisis de los escenarios urbanos generados durante las simulaciones.

6.3.1 Qwen2.5-VL

El primer modelo evaluado fue **Qwen2.5-VL**, un modelo multimodal de código abierto desarrollado por Alibaba y orientado a tareas de comprensión visual. Para los experimentos se empleó la variante de 7 mil millones de parámetros (7B), ejecutada localmente mediante LM Studio, lo que permitió realizar todas las inferencias sin depender de servicios externos y mantener unas condiciones experimentales homogéneas durante toda la evaluación.

La interacción con el modelo se realizó a través de la API compatible con OpenAI proporcionada por LM Studio, integrada en una aplicación desarrollada en Python. Para cada fotograma seleccionado se enviaba una petición compuesta por el *system prompt* descrito en la Sección 6.2.2, un *user prompt* encargado de concretar la tarea a realizar en cada inferencia, y las cuatro imágenes correspondientes al instante analizado: la escena completa y las tres regiones obtenidas al dividir la imagen en las zonas izquierda, central y derecha. El *user prompt* indicaba al modelo que analizara la intersección, determinara el estado de las vías situadas a la izquierda, al frente y a la derecha del vehículo, y devolviera exclusivamente una respuesta en formato JSON, complementando así las instrucciones generales del *system prompt* con el objetivo específico de cada inferencia.

Con el objetivo de obtener respuestas deterministas y facilitar la comparación entre ejecuciones, la temperatura del modelo se fijó en 0, eliminando la variabilidad introducida por el proceso de generación. Asimismo, el número máximo de tokens de salida se limitó a 300, cantidad suficiente para producir la respuesta estructurada en formato JSON sin generar información adicional innecesaria.

Tabla 6.2: Parámetros de inferencia utilizados para Qwen2.5-VL.

Parámetro	Valor
Modelo	Qwen2.5-VL-7B
Entorno de ejecución	LM Studio (local)
API	Compatible con OpenAI
Temperatura	0.0
Máximo de tokens	300

6.3.2 LLaVA 1.6

El segundo modelo evaluado fue **LLaVA 1.6**, un modelo multimodal de código abierto que combina un codificador visual con un modelo de lenguaje basado en Vicuna para realizar tareas de comprensión de imágenes y generación de texto. Para los experimentos se empleó la variante de 7 mil millones de parámetros (7B), ejecutada localmente mediante LM Studio, al igual que el resto de modelos evaluados.

La configuración empleada fue idéntica a la utilizada con Qwen2.5-VL, manteniendo los mismos parámetros de inferencia, formato de entrada, conjunto de fotogramas analizados y *prompt* descrito en la Sección 6.2.2, con el fin de garantizar unas condiciones experimentales homogéneas que permitieran una comparación objetiva entre modelos.

Durante las primeras pruebas se observó que LLaVA 1.6 presentaba mayores dificultades que otros modelos evaluados para interpretar correctamente determinados escenarios, especialmente en la identificación de bloqueos y en la localización espacial de elementos dentro de la escena, lo que se tradujo en una menor consistencia de las respuestas obtenidas.

6.3.3 InternVL3

El tercer modelo evaluado fue **InternVL3**, un modelo multimodal de código abierto desarrollado por OpenGVLab y diseñado para la comprensión conjunta de información visual y textual. Para los experimentos se empleó la variante de 8 mil millones de parámetros (8B), ejecutada localmente mediante LM Studio bajo las mismas condiciones generales que el resto de modelos evaluados.

Al igual que en los casos anteriores, se mantuvieron el *system prompt* y el *user prompt* descritos en la Sección 6.2.2. No obstante, fue necesario realizar ajustes en algunos parámetros de inferencia con el fin de optimizar el comportamiento del modelo. La temperatura se incrementó hasta 0,5 para favorecer una mayor capacidad de razonamiento sin comprometer la estabilidad de las respuestas, y el parámetro *top-p* se fijó en 0,4 para restringir la selección de tokens a las opciones más probables. Asimismo, el número máximo de tokens generados se amplió hasta 512 para permitir respuestas más completas, y se estableció una semilla (*seed*) de valor 42 para garantizar la reproducibilidad de los experimentos.

Desde las primeras pruebas, InternVL3 mostró un comportamiento más consistente en la interpretación de los escenarios en comparación con los modelos anteriores, con una menor aparición de respuestas inconsistentes y de errores en la identificación de obstáculos.

Tabla 6.3: Parámetros de inferencia utilizados para InternVL3.

Parámetro	Valor
Modelo	InternVL3-8B
Entorno de ejecución	LM Studio (local)
Temperatura	0.5
Top- <i>p</i>	0.4
Máximo de tokens	512
Semilla (<i>seed</i>)	42

6.3.4 Qwen3-VL

El último modelo evaluado fue **Qwen3-VL**, sucesor de Qwen2.5-VL y desarrollado igualmente por Alibaba. Para los experimentos se empleó la variante de 8 mil millones de parámetros (8B), ejecutada localmente mediante LM Studio bajo las mismas condiciones generales que el resto de modelos evaluados.

Al igual que en los casos anteriores, se mantuvieron el *system prompt* y el *user prompt* descritos en la Sección 6.2.2. No obstante, se ajustaron algunos parámetros de inferencia con el objetivo de adaptar el comportamiento del modelo a las características de la tarea. La temperatura se fijó en 0,2 y el parámetro *top-p* en 0,2, favoreciendo una generación más determinista y reduciendo la variabilidad de las respuestas. Asimismo, el número máximo de tokens se estableció en 400 y se empleó una semilla de valor 30 para garantizar la reproducibilidad de los experimentos.

Durante las pruebas preliminares, Qwen3-VL mostró un comportamiento consistente en la interpretación de los escenarios simulados, con una identificación más precisa de los bloqueos presentes en la vía y una menor aparición de respuestas ambiguas.

Tabla 6.4: Parámetros de inferencia utilizados para Qwen3-VL.

Parámetro	Valor
Versión del modelo	Qwen3-VL-8B
Temperatura	0.2
Top- p	0.2
Máximo de tokens	400
Semilla (<i>seed</i>)	30

6.4 Evaluación realizada sobre los modelos

6.4.1 Diseño de las pruebas

El objetivo principal de los experimentos fue evaluar la capacidad de los distintos modelos multimodales para interpretar escenarios urbanos simulados e identificar correctamente la presencia de bloqueos que modificaban la trayectoria prevista del vehículo autónomo. Para ello, todos los modelos fueron evaluados utilizando exclusivamente recorridos en los que existía al menos un obstáculo situado sobre la vía de circulación, de forma que el vehículo se veía obligado a alterar la ruta inicialmente planificada.

Durante el análisis, los modelos debían determinar si la vía situada en cada una de las direcciones evaluadas (izquierda, frente y derecha) era transitable o se encontraba bloqueada. Para considerar que un bloqueo había sido identificado correctamente, el modelo debía reconocer la presencia de un obstáculo físico localizado dentro de la zona transitable de la calzada e impidiendo el paso del vehículo. Además, se esperaba que el modelo fuera capaz de identificar el tipo de elemento responsable del bloqueo e interpretar la escena de forma coherente, diferenciando entre obstáculos reales y otros elementos del entorno urbano, como señales de tráfico, edificios, aceras o mobiliario urbano, que no afectaban directamente a la circulación.

Con el fin de cubrir diferentes situaciones de conducción, se seleccionaron rutas pertenecientes tanto a los escenarios predeterminados de [CARLA](#) como a los escenarios personalizados desarrollados a partir de datos de OpenStreetMap. Para cada entorno se generaron aproximadamente cuatro recorridos con distintos tipos de bloqueos y variaciones en la trayectoria del vehículo, obteniendo un conjunto de escenarios suficientemente variado para analizar la capacidad de generalización de los modelos frente a diferentes configuraciones urbanas y tipos de obstáculos.

Tabla 6.5: Resumen de los escenarios utilizados durante la evaluación experimental.

Característica	Valor
Escenarios predeterminados de CARLA	3
Escenarios generados desde OpenStreetMap	2
Total de escenarios evaluados	5
Rutas por escenario	≈ 4
Vídeos analizados	22

En la siguiente tabla se resumen los criterios de evaluación de los modelos.

Tabla 6.6: Criterios empleados para la evaluación de los modelos.

Aspecto evaluado	Descripción	Resultado esperado
Existencia de bloqueo	Determinar si existe un obstáculo físico que impida completamente la circulación del vehículo por una determinada dirección.	El modelo identifica correctamente la presencia o ausencia de un bloqueo en la vía analizada.
Localización del bloqueo	Identificar la dirección (izquierda, frente o derecha) en la que se encuentra el obstáculo.	La dirección indicada coincide con la posición real del bloqueo en el escenario.
Tipo de obstáculo	Reconocer el elemento responsable del bloqueo (vehículo, barreras, conos, obras, etc.).	El modelo clasifica correctamente el objeto que provoca el bloqueo.
Interpretación del entorno	Distinguir entre obstáculos reales y elementos del entorno que no afectan a la circulación.	El modelo ignora señales, edificios, vegetación, aceras y otros elementos situados fuera de la trayectoria del vehículo.
Nivel de confianza	Asignar un valor de confianza a cada predicción realizada.	El nivel de confianza es coherente con la claridad de la escena y la certeza de la predicción.
Justificación	Explicar brevemente el motivo de la decisión adoptada para cada dirección analizada.	La explicación es consistente con el contenido visual del escenario y con la clasificación realizada.

6.5 Resultados obtenidos

Una vez completada la preparación de los datos y configurados los distintos modelos, se procedió a la ejecución de los experimentos y al análisis de los resultados obtenidos. En una primera fase se realizó una evaluación preliminar con el objetivo de comparar el comportamiento de los distintos modelos bajo las mismas condiciones experimentales y seleccionar aquel que ofreciera el mejor rendimiento para la tarea de análisis semántico de los escenarios.

Para ello, se emplearon métricas clásicas utilizadas en problemas de clasificación y detección, las cuales permitieron cuantificar de forma objetiva la capacidad de cada modelo para identificar correctamente los bloqueos presentes en las distintas rutas y valorar su adecuación para la evaluación de los escenarios modificados y generados durante el desarrollo.

6.5.1 Evaluación preliminar de los modelos

Antes de realizar una evaluación exhaustiva sobre el conjunto completo de escenarios, se llevó a cabo una fase preliminar cuyo objetivo fue comparar el comportamiento de los distintos modelos multimodales bajo las mismas condiciones experimentales. Para ello se seleccionó un conjunto representativo de cuatro vídeos que incluían diferentes configuraciones de bloqueos, permitiendo obtener una primera estimación del rendimiento de cada modelo en la tarea de análisis semántico del entorno.

La evaluación se realizó empleando las métricas Precision, Recall y F1-Score, ampliamente utilizadas en problemas de clasificación y detección. La métrica Precision cuantifica la proporción de bloqueos detectados correctamente respecto al total de detecciones realizadas por el modelo; Recall mide la capacidad del modelo para identificar los bloqueos realmente presentes en los escenarios; y el F1-Score combina ambas en un único indicador, proporcionando una medida equilibrada del rendimiento general. Sus expresiones de cálculo se muestran en las ecuaciones 6.1, 6.2 y 6.3, donde los True Positives (TP) representan los bloqueos correctamente detectados, los False Positives (FP) corresponden a falsas detecciones de bloqueos inexistentes y los False Negatives (FN) representan bloqueos presentes en el escenario que no fueron identificados por el modelo.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6.1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6.2)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6.3)$$

Los resultados obtenidos durante esta evaluación preliminar se recogen en la Tabla 6.7. Qwen3-VL obtuvo el mejor rendimiento entre todos los modelos evaluados, alcanzando una Precision del 90,9%, un Recall del 100% y un F1-Score del 95,2%, detectando correctamente la totalidad de los bloqueos presentes en los escenarios analizados y generando únicamente una falsa detección. Dado que este modelo demostró el mayor equilibrio entre precisión y exhaustividad, fue seleccionado para realizar la evaluación completa sobre el conjunto de los 22 vídeos empleados en este trabajo.

Tabla 6.7: Resultados obtenidos durante la evaluación preliminar de los modelos multimodales.

Modelo	TP	FP	FN	Precision	Recall	F1-Score
Qwen2.5-VL	5	4	5	55.6%	50.0%	52.6%
InternVL3	7	2	3	77.8%	70.0%	73.7%
LLaVA 1.6	4	6	6	40.0%	40.0%	40.0%
Qwen3-VL	10	1	0	90.9%	100.0%	95.2%

6.5.2 Evaluación completa del modelo seleccionado

Tras la fase de evaluación preliminar, se seleccionó **Qwen3-VL** para realizar la evaluación completa sobre el conjunto de datos generado durante el desarrollo del trabajo, en base a los resultados obtenidos en dicha fase, donde este modelo alcanzó los mejores valores de *Precision*, *Recall* y *F1-Score*.

La evaluación completa se llevó a cabo utilizando los **22 vídeos** correspondientes a las diferentes rutas diseñadas tanto en los escenarios predeterminados de CARLA como en los escenarios personalizados generados a partir de OpenStreetMap. En total, el conjunto de datos contenía **50 bloqueos reales**, distribuidos entre los distintos recorridos y tipos de obstáculos empleados durante las simulaciones.

De los 50 bloqueos presentes, el modelo identificó correctamente **40**, produciendo **9 falsas detecciones** y dejando sin identificar **10 bloqueos**. Adicionalmente, se registraron **5 respuestas clasificadas como *unknown***, correspondientes a situaciones en las que el modelo no fue capaz de determinar con suficiente confianza el estado de la vía.

A partir de estos resultados se calcularon las métricas de evaluación empleadas durante el estudio, cuyos valores se presentan en la Tabla 6.8. El modelo alcanzó una *Precision* del **81,6%**, un *Recall* del **80,0%** y un *F1-Score* del **80,8%**, confirmando un comportamiento consistente sobre un conjunto de escenarios significativamente más amplio y variado que el utilizado en la evaluación preliminar.

Tabla 6.8: Resultados de la evaluación completa realizada con Qwen3-VL.

Indicador	Valor
Vídeos analizados	22
Bloqueos reales	50
True Positives (TP)	40
False Positives (FP)	9
False Negatives (FN)	10
Precision	81.6%
Recall	80.0%
F1-Score	80.8%



(a) Modelo identifica correctamente bloqueo.



(b) Modelo no detecta correctamente calle izquierda bloqueada.

Figura 6.3: Pruebas de identificación de obstáculos del modelo elegido.

6.5.3 Análisis cualitativo

Una vez presentados los resultados cuantitativos obtenidos durante la evaluación completa, resulta conveniente complementarlos con un análisis cualitativo del comportamiento mostrado por Qwen3-VL en los distintos escenarios evaluados, con el fin de identificar tanto las fortalezas del modelo como las principales limitaciones observadas durante las pruebas.

En términos generales, el modelo detectó correctamente la mayoría de los bloqueos presentes en las rutas analizadas, tal y como reflejan las métricas presentadas en el apartado anterior. Los mejores resultados se obtuvieron en escenarios donde los obstáculos estaban formados por elementos claramente identificables y habitualmente asociados al corte o restricción del tráfico, como barreras, conos de señalización, paneles luminosos de obras o señales de advertencia. Del mismo modo, el modelo mostró una elevada capacidad para reconocer bloqueos provocados por vehículos de emergencia, como ambulancias, coches patrulla o camiones de bomberos, así como por elementos de gran tamaño situados directamente sobre la calzada, como contenedores u otros obstáculos de dimensiones similares.

Las principales dificultades aparecieron en escenarios donde el bloqueo estaba compuesto por varios elementos de menor tamaño o por obstáculos parcialmente visibles debido a la perspectiva de la cámara o a la distancia respecto al vehículo. En estas situaciones, combinaciones de elementos como muros de piedra, cadenas de acceso o vallas metálicas no siempre fueron interpretadas como un bloqueo completo de la vía, provocando que algunos obstáculos reales no fueran detectados.

Se observaron también algunos casos aislados de falsas detecciones. En determinadas escenas, el modelo clasificó erróneamente elementos como semáforos o señales situadas junto a la calzada como posibles bloqueos, a pesar de no interferir con la trayectoria del vehículo. Este comportamiento puede atribuirse a las limitaciones derivadas de la perspectiva de la cámara y a la reducida diferencia visual entre la calzada y el bordillo de la acera, especialmente cuando ambos aparecen próximos dentro del campo de visión.

Por último, se registraron varias respuestas clasificadas como *unknown*, correspondientes a escenas en las que el modelo consideró que no disponía de información visual suficiente para determinar con certeza el estado de una determinada dirección. Este comportamiento no constituye un error de funcionamiento, sino una decisión conservadora que evita emitir clasificaciones cuando el nivel de incertidumbre es elevado. No obstante, una reducción de estas respuestas mediante un refinamiento adicional del *prompt* o mediante el ajuste de los parámetros de inferencia podría contribuir a mejorar el rendimiento global del modelo en futuros trabajos.

6.6 Interpretación de los resultados

Los resultados obtenidos durante la fase experimental permiten afirmar que los modelos multimodales actuales poseen un elevado potencial para realizar tareas de análisis semántico sobre entornos de conducción simulados. No obstante, el rendimiento observado varía significativamente en función del modelo empleado, lo que pone de manifiesto la importancia de seleccionar adecuadamente la arquitectura utilizada para este tipo de aplicaciones.

La evaluación preliminar mostró diferencias notables entre los cuatro modelos analizados. Mientras que LLaVA 1.6 presentó un mayor número de falsas detecciones y dificultades para distinguir entre elementos pertenecientes a la calzada y objetos situados en sus proximidades, InternVL3 y especialmente Qwen3-VL ofrecieron un comportamiento considerablemente más consistente. Esta diferencia puede atribuirse a una mejor capacidad para comprender el contexto espacial de la escena y relacionar la información visual con el razonamiento necesario para determinar si una vía se encontraba realmente bloqueada.

Los resultados de la evaluación completa confirmaron esta tendencia. Qwen3-VL detectó correctamente la mayoría de los bloqueos presentes en los distintos escenarios, alcanzando valores equilibrados de *Precision* y *Recall*. El análisis cualitativo reveló además que el modelo era especialmente eficaz cuando los obstáculos estaban claramente representados mediante elementos de señalización vial o vehículos de gran tamaño, mientras que continuó mostrando limitaciones en escenarios con bloqueos compuestos por varios elementos pequeños, parcialmente ocultos o situados a una distancia considerable del vehículo.

Otro aspecto destacable fue la influencia del diseño del *prompt* sobre el comportamiento de los modelos. El proceso iterativo de refinamiento permitió reducir ambigüedades, limitar las falsas detecciones y mejorar la consistencia de las respuestas generadas. Asimismo, la división de cada fotograma en regiones espaciales correspondientes a la izquierda, el centro y la derecha facilitó la interpretación de la escena, reduciendo los errores de localización observados durante las primeras pruebas.

No obstante, el trabajo presenta diversas limitaciones que deben tenerse en consideración. Los experimentos se realizaron íntegramente sobre escenarios simulados, por lo que las condiciones de iluminación, meteorología y comportamiento del tráfico se encuentran controladas y no reflejan la variabilidad presente en entornos reales. Además, el análisis se llevó a cabo utilizando únicamente la información proporcionada por una cámara RGB frontal, sin incorporar otros sensores habitualmente presentes en sistemas de conducción autónoma, como LiDAR, radar o cámaras adicionales.

A pesar de estas limitaciones, los resultados demuestran que los modelos pueden constituir una herramienta de gran utilidad para la interpretación automática de escenarios urbanos simulados. Su capacidad para identificar bloqueos, describir el entorno y proporcionar una interpretación semántica de las escenas abre nuevas posibilidades para el desarrollo de herramientas de validación automática de simulaciones, asistencia al análisis de escenarios y apoyo a futuras investigaciones en el ámbito de la conducción autónoma.

Capítulo 7

Conclusiones y líneas futuras

7.1 Conclusiones

El presente Trabajo Fin de Grado ha tenido como objetivo principal el desarrollo de una metodología integral para la creación, modificación y evaluación de entornos urbanos simulados, orientada al estudio de modelos multimodales aplicados a la conducción autónoma. Para ello, se ha diseñado un flujo de trabajo que integra diferentes herramientas y tecnologías, permitiendo generar escenarios de forma flexible, incorporar bloqueos que alteran las condiciones normales de circulación y evaluar la capacidad de distintos modelos de lenguaje con comprensión visual para interpretar dichas situaciones.

Los objetivos planteados al inicio del trabajo se han alcanzado satisfactoriamente. Por un lado, se ha desarrollado un procedimiento que permite modificar los escenarios predeterminados de CARLA mediante la incorporación de distintos tipos de obstáculos distribuidos en múltiples ubicaciones de cada mapa, generando situaciones de circulación más próximas a las que pueden encontrarse en un entorno urbano real. De forma complementaria, se ha implementado un proceso de generación semiautomatizada de nuevos escenarios urbanos a partir de información cartográfica de OpenStreetMap que, mediante RoadRunner y su posterior integración en CARLA, permite construir entornos basados en ubicaciones reales sin necesidad de diseñar manualmente nuevos mapas desde cero.

Una de las principales aportaciones del trabajo es la infraestructura desarrollada para la definición y ejecución de experimentos. El uso de archivos de configuración en formato YAML permite definir la posición de los obstáculos, las rutas de los vehículos y el resto de parámetros de cada simulación sin necesidad de modificar el código de la aplicación, lo que facilita la incorporación de nuevos escenarios y hace del sistema una herramienta reutilizable para futuras investigaciones.

Asimismo, se ha diseñado una metodología para la obtención y preparación de los datos destinados a los modelos multimodales, que comprende la captura automática de vídeo desde el simulador, su procesamiento para facilitar la interpretación espacial de las escenas y el desarrollo de un *prompt* general mediante un proceso iterativo de refinamiento. El resultado fue un *prompt* suficientemente robusto para todos los modelos evaluados, lo que sugiere que las diferencias de rendimiento observadas estuvieron más relacionadas con las características propias de cada arquitectura que con la formulación de las instrucciones proporcionadas.

En la fase experimental, Qwen3-VL fue seleccionado tras una primera evaluación comparativa, obteniendo los mejores resultados tanto en las métricas cuantitativas como en el análisis cualitativo. El modelo demostró una elevada capacidad para identificar bloqueos sobre la calzada, distinguir entre

obstáculos reales y elementos del entorno urbano, y proporcionar respuestas consistentes incluso en situaciones con cierto grado de incertidumbre.

Más allá de la comparación entre modelos, la principal contribución de este trabajo reside en la propia metodología experimental, que abarca todas las etapas necesarias para este tipo de estudios: desde la generación y modificación de escenarios urbanos hasta la evaluación sistemática de modelos multimodales, pasando por la automatización de rutas y bloqueos y la obtención y preparación de la información visual. Esta infraestructura constituye una plataforma reutilizable que facilita la incorporación de nuevos escenarios, rutas y modelos sin necesidad de reconstruir el proceso experimental completo.

En conjunto, los resultados obtenidos ponen de manifiesto que los modelos multimodales actuales poseen un notable potencial para la interpretación semántica de entornos de conducción simulados. Aunque todavía presentan limitaciones en situaciones complejas, su capacidad para comprender escenas urbanas y detectar bloqueos demuestra que pueden convertirse en herramientas de gran utilidad como apoyo al desarrollo, validación y evaluación de futuras tecnologías de conducción autónoma.

Por tanto, puede concluirse que este trabajo no solo ha permitido alcanzar los objetivos inicialmente propuestos, sino que además proporciona una base sólida sobre la que continuar investigando la aplicación de modelos multimodales en simulación urbana, reduciendo el esfuerzo necesario para generar nuevos escenarios de prueba y facilitando la evaluación sistemática de futuros modelos en el ámbito de la percepción inteligente y la conducción autónoma.

7.2 Líneas futuras

El trabajo desarrollado constituye una base sólida para nuevas investigaciones relacionadas con la generación de escenarios urbanos simulados y la evaluación de modelos multimodales aplicados a la conducción autónoma. A partir de esta plataforma, se identifican varias líneas de trabajo que permitirían ampliar sus capacidades y mejorar los resultados obtenidos.

En primer lugar, cabría ampliar el conjunto de modelos evaluados. Dada la rápida evolución de este tipo de tecnologías, sería recomendable incorporar futuras versiones de los modelos analizados, así como nuevas arquitecturas que vayan apareciendo, aprovechando que la infraestructura desarrollada permite realizar estas comparativas sin modificar el resto del flujo experimental.

Otra línea de trabajo consistiría en incrementar la complejidad de los escenarios, incorporando situaciones dinámicas como accidentes de tráfico, obras en curso, vehículos averiados o peatones que interfieran con la circulación, además de ampliar el número de mapas generados a partir de datos reales de OpenStreetMap para incrementar la diversidad de entornos evaluados.

De forma complementaria, resultaría útil estudiar el comportamiento de los modelos bajo distintas condiciones ambientales —lluvia, niebla, conducción nocturna o baja visibilidad—, así como enriquecer la información proporcionada mediante sensores adicionales, como LiDAR, radar o cámaras laterales, que permitan una representación más completa del entorno.

Asimismo, la metodología podría evolucionar hacia el análisis de secuencias completas de vídeo incorporando razonamiento temporal, lo que permitiría interpretar con mayor precisión la evolución de los escenarios y detectar eventos dinámicos a lo largo de la ruta.

Por último, una línea especialmente interesante consiste en integrar esta plataforma con sistemas de planificación y conducción autónoma, empleando la información semántica generada por los modelos como apoyo para la toma de decisiones, la validación automática de rutas o la detección de incidencias durante la simulación.

En definitiva, la infraestructura desarrollada en este Trabajo Fin de Grado ofrece un entorno modular y fácilmente ampliable, capaz de extender cada una de sus fases —desde la generación de escenarios hasta la evaluación de modelos multimodales— con un elevado potencial para futuras investigaciones en simulación urbana, visión por computador e inteligencia artificial aplicada a la conducción autónoma.

Bibliografía

- [1] SAE International, *Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles*, 2021. dirección: <https://www.sae.org/>.
- [2] E. Yurtsever, J. Lambert, A. Carballo y K. Takeda, “A Survey of Autonomous Driving: Common Practices and Emerging Technologies”, *IEEE Access*, 2020, mar. de 2020.
- [3] H. Alemayehu y A. Sargolzaei, “Testing and Verification of Connected and Autonomous Vehicles: A Review”, *Electronics*, feb. de 2025.
- [4] C. Li, J. Sifakis, Q. Wang, R. Yan y J. Zhang, “Simulation-Based Validation for Autonomous Driving Systems”, en *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023.
- [5] E. Thorn, S. Kimmel, M. Chaka et al., “A framework for automated driving system testable cases and scenarios”, United States. Department of Transportation. National Highway Traffic Safety Administration, inf. téc., 2018.
- [6] W. Zhao, J. P. Queralta y T. Westerlund, “Sim-to-real transfer in autonomous driving: A review”, *arXiv preprint arXiv:2004.14376*, 2020.
- [7] Microsoft, *AirSim official documentation*, 2021. dirección: <https://microsoft.github.io/AirSim/>.
- [8] LG Electronics Silicon Valley Lab (LGSVL), *SVL simulator: running the SVL simulator alongside Autoware.Auto*, 2022. dirección: <https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/lgsvl.html>.
- [9] Andrew Howard, University of Southern California, *Gazebo simulator*, 2025. dirección: <https://gazebo.org/home>.
- [10] Computer Vision Center, Barcelona, *CARLA official documentation*, 2025. dirección: <https://carla.readthedocs.io/en/latest/>.
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah et al., “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems*, 2020.
- [12] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, *Advances in Neural Information Processing Systems*, 2022.
- [13] S. Yin et al., “A Survey on Multimodal Large Language Models”, *National Science Review*, 2024.
- [14] J.-B. Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning”, *Advances in Neural Information Processing Systems*, 2022.
- [15] OpenAI, *GPT-4 Technical Report*, 2023.
- [16] W. X. Zhao et al., “A Survey of Large Language Models”, *arXiv preprint arXiv:2303.18223*, 2023. DOI: [10.48550/arXiv.2303.18223](https://arxiv.org/abs/2303.18223). dirección: <https://arxiv.org/abs/2303.18223>.

- [17] J. Zhang, J. Huang, S. Jin y S. Lu, “Vision-Language Models for Vision Tasks:A Survey”, *IEEE Transactions on Pattern Analysis and Machine Intelligence.*, 2024.
- [18] X. Zhou et al., “Vision Language Models in Autonomous Driving: A Survey and Outlook”, *Computer Vision and Pattern Recognition*, 2024.

Apéndice A

Detalles de implementación

Este anexo tiene como objetivo presentar los principales archivos de configuración empleados durante el desarrollo de este trabajo, así como los *prompts* utilizados para la interacción con los modelos de lenguaje y ejemplos de las respuestas generadas por estos. Además, se incluyen fragmentos de código y comandos de ejecución relevantes que complementan la información expuesta en el cuerpo principal del documento.

A.1 Comandos de ejecución

```
1 python Util\BuildTools\Import.py ^  
2 --package=escenario ^  
3 --input=C:\carla\Import\escenario\escenario ^  
4 --no-carla-materials
```

Listado A.1: Comando de importación de entornos en CARLA (Windows CMD)

```
1 ffmpeg -framerate 10 -i %06d.png -c:v libx264 -pix_fmt yuv420p -crf 18 output.mp4
```

Listado A.2: Comando de FFmpeg para generar video a partir de imágenes PNG

A.2 Archivos de configuración y archivos de LLM

```
1 {  
2   "maps": [  
3     {  
4       "name": "Madrid",  
5       "source": "./madrid_map/Town_test_rd.fbx",  
6       "use_carla_materials": false,  
7       "xodr": "./madrid_map/Town_test_rd.xodr"  
8     }  
9   ],  
10  "props": [  
11  ]  
12 }
```

Listado A.3: Ejemplo de archivo necesario para importar mapas personalizados a CARLA

```

1 scenarios:
2
3   blocked_street01_Town02: #roadblock type identifier
4
5   map: "Town02"
6   description: |
7       First roadblock consisting of a single closed street in Town02.
8       This roadblock consists of 2 road barriers (static.prop.streetbarrier)
9       and 2 traffic cones (static.prop.trafficcone01),
10      simulating a roadblock due to construction work.
11
12   props: #List of props
13       static.prop.streetbarrier: #Props ID
14
15       - transform: #coordinates and rotations for generating props
16         location: {x: 45.91, y: 250.181, z: 0.02}
17         rotation: {pitch: 0.0 , yaw: 0.0 , roll: 0.0}
18
19       - transform:
20         location: {x: 41.80 , y: 250.20 ,z: 0.02}
21         rotation: {pitch: 0.0 , yaw: 0.0, roll: 0.02 }
22
23   static.prop.trafficcone01:
24
25       - transform:
26         location: {x: 47.19 , y: 250.44 , z: 0.02 }
27         rotation: {pitch: 0.0 , yaw: 90.0 , roll: 0.0}
28
29       - transform:
30         location: {x: 40.60 ,y: 250.40,z: 0.03}
31         rotation: {pitch: 0.0 , yaw: 90.0 , roll: 0.0}
32
33   static.prop.trafficcone02:
34
35       - transform: #Route reference
36         location: {x: 125.30 ,y: 238.70,z: 7.30}
37         rotation: {pitch: 90.0 , yaw: 0.0 , roll: 0.0}
38
39       - transform: #Route reference
40         location: {x: 124.20 ,y: 238.70,z: 7.30}
41         rotation: {pitch: 90.0 , yaw: 0.0 , roll: 0.0}
42
43       - transform: #Route reference
44         location: {x: 123.80 ,y: 239.50,z: 7.30}
45         rotation: {pitch: 90.0 , yaw: 0.0 , roll: -45.0}
46
47       - transform: #Route reference
48         location: {x: 123.90 ,y: 237.90,z: 7.30}
49         rotation: {pitch: 90.0 , yaw: 0.0 , roll: 45.0}
50
51   static.prop.stop:
52
53       - transform: #Route reference
54         location: {x: 72.80 ,y: 304.00,z: 7.30}
55         rotation: {pitch: 0.0 , yaw: 90.0 , roll: 0.0}

```

Listado A.4: Ejemplo completo de archivo de configuración para la generación dinámica de obstáculos

```

1 {
2   "props": [
3     {
4       "name": "ATM",
5       "path": "/Game/Carla/Static/Static/SM_Atm.SM_Atm",
6       "size": "Medium"
7     },
8     {
9       "name": "Bin",
10      "path": "/Game/Carla/Static/Dynamic/Trash/SM_Bin.SM_Bin",
11      "size": "Medium"
12    },
13    {
14      "name": "Box01",
15      "path": "/Game/Carla/Static/Dynamic/Trash/SM_Box01.SM_Box01",
16      "size": "Small"
17    },
18    {
19      "name": "Box02",
20      "path": "/Game/Carla/Static/Dynamic/Trash/SM_box02.SM_box02",
21      "size": "Small"
22    },
23    {
24      "name": "Box03",
25      "path": "/Game/Carla/Static/Dynamic/Trash/SM_box03.SM_box03",
26      "size": "Small"
27    },
28    {
29      "name": "Briefcase",
30      "path": "/Game/Carla/Static/Dynamic/PedestrianProps/SM_Briefcase.SM_Briefcase",
31      "size": "Small"
32    },
33    {
34      "name": "BrokenTile01",
35      "path": "/Game/Carla/Static/Dynamic/Trash/Wastes/SM_BrokenTile01.SM_BrokenTile01",
36      "size": "Tiny"
37    },
38    {
39      "name": "BrokenTile02",
40      "path": "/Game/Carla/Static/Dynamic/Trash/Wastes/SM_BrokenTile02.SM_BrokenTile02",
41      "size": "Tiny"
42    },
43    {
44      "name": "BrokenTile03",
45      "path": "/Game/Carla/Static/Dynamic/Trash/Wastes/SM_BrokenTile03.SM_BrokenTile03",
46      "size": "Tiny"
47    },
48    {
49      "name": "Stop",
50      "path": "/Game/Carla/Static/TrafficSign/Stop/Stop_v01/SM_stopSign.SM_stopSign",
51      "size": "Medium"

```

```

52
53     }
54 ]
55 }

```

Listado A.5: Ejemplo del catálogo Default.package.json de props generables por medio de la API y añadidos

```

1 {
2   "left_street": {
3     "blocked": false,
4     "confidence": 0.92,
5     "reason": "path appears clear with no physical obstructions"
6   },
7   "front_street": {
8     "blocked": true,
9     "confidence": 0.95,
10    "reason": "construction barriers blocking forward path"
11  },
12  "right_street": {
13    "blocked": false,
14    "confidence": 0.85,
15    "reason": "grass field with no visible blockage"
16  },
17  "frame": 5
18 }

```

Listado A.6: Formato de respuestas obtenidas de modelos LLM

A.3 Prompts utilizados para los modelos probados

```

1 You are an autonomous driving scene analyzer.
2
3 Your ONLY task is to determine whether streets visible at an intersection are physically blocked.
4
5 You will receive:
6
7 1. Full driving scene image
8 2. LEFT region crop
9 3. CENTER region crop
10 4. RIGHT region crop
11
12 Interpretation:
13
14 - LEFT crop = left street/path
15 - CENTER crop = forward street/path
16 - RIGHT crop = right street/path
17
18 A street is BLOCKED only if a physical obstacle prevents vehicle passage.
19
20 Examples of valid blockage:
21 - road barriers
22 - construction barricades

```

```

23 - fences
24 - crashed vehicles blocking the road
25 - cones or cone poles completely closing the street
26 - police, ambulance or fire trucks barricades
27 - trucks in discharge operations
28 - Trucks in the road
29 - Light Traffic Signals
30 - White pole cones
31 - Car accidents
32
33 DO NOT consider as blockage:
34 - traffic lights
35 - stop signs
36 - pedestrians
37 - shadows
38 - lane markings
39 - parked vehicles unless fully blocking passage
40
41 IMPORTANT:
42 - Be conservative.
43 - Only report blocked=true if blockage is visually clear.
44 - If visibility is insufficient return blocked="unknown".
45
46 Return ONLY valid JSON.
47
48 JSON schema:
49
50 {
51   "left_street": {
52     "blocked": true,
53     "confidence": 0.95,
54     "reason": "construction barriers"
55   },
56   "front_street": {
57     "blocked": false,
58     "confidence": 0.88,
59     "reason": "street appears drivable"
60   },
61   "right_street": {
62     "blocked": "unknown",
63     "confidence": 0.40,
64     "reason": "insufficient visibility"
65   }
66 }

```

Listado A.7: Prompt de sistema utilizado para guiar el comportamiento del modelo LLM

```

1 Analyze the intersection.
2
3 Determine whether the LEFT, FRONT, and RIGHT streets are physically blocked.
4
5 Use the spatial crops to identify each direction correctly.
6
7 Return JSON only.

```

Listado A.8: Ejemplo de prompt de usuario enviado al modelo LLM

Apéndice B

Herramientas y recursos

Las herramientas necesarias para la elaboración del proyecto han sido:

- PC compatible
- Sistema operativo GNU/Linux o Windows
- Editor de código Visual Studio Code
- Tarjeta gráfica de al menos 8 GB de memoria dedicada VRAM
- Editor interactivo RoadRunner
- Motor gráfico Unreal Engine versión 4.26
- Python versiones 3.11 en adelante
- Editor de contenido audiovisual FFmpeg
- LM Studio
- Modelo Qwen3-VL 8B
- Modelo InternVL3 8B
- Modelo LLaVA 1.6 7B
- Modelo Qwen2.5-VL 7B
- Simulador CARLA



Universidad
de Alcalá

2026